UNIVERSITÀ DEGLI STUDI DI PISA

Facoltà di Scienze Matematiche Fisiche e Naturali

Corso di Laurea in Informatica

Realizzazione di una versione Muskel/FastFlow

Candidato
Daniele De Sensi

Relatore
Prof. Marco Danelutto

Anno Accademico 2009-2010

Ai miei cari

Indice

Elenco delle figure **vi**

Elenco dei codici **vii**

Introduzione **viii**

1 Programmazione a skeleton **1**
1.1 Skeletons stream parallel 3
1.1.1 Farm 3
1.1.2 Pipeline 4
1.2 Skeletons data parallel 5
1.2.1 Map 5
1.2.2 Reduce 6
1.3 Sequenziale 7

2 Strumenti utilizzati **9**
2.1 C++ 9
2.2 FastFlow 11

3 Faskel: struttura ed implementazione **15**
3.1 Task e streams 15
3.2 Grafi macro data flow 18
3.3 Implementazione ed utilizzo degli skeleton 20
3.3.1 Computable 22
3.3.2 Farm 23
3.3.3 Pipeline 24
3.3.4 Map e reduce 26
3.4 Compilatore 27
3.5 Interprete 28
3.6 Manager 29

4 Risultati sperimentali **34**
4.1 Test funzionalità 34
4.2 Test scalabilità 35
4.3 Accorgimenti adottati al fine di aumentare le prestazioni . . . 36
4.4 ffProbe: realizzazione di una cache NetFlow 39
4.4.1 Struttura con FastFlow 40
4.4.2 Struttura con Faskel 43
4.4.3 Test 43

5 Conclusioni e sviluppi futuri **44**

Bibliografia **46**

Elenco delle figure

1.1 Farm con 4 workers 3
1.2 Pipeline a 3 stadi 4
1.3 Map con 4 workers 5
1.4 Reduce con 4 workers 7
1.5 Composizione di skeleton 8

2.1 Architettura di FastFlow 11
2.2 Esempi di grafi definibili utilizzando le code SPSC. 13

3.1 Esempio di un grafo macro data flow 18
3.2 Interazione manager-interprete 29
3.3 Funzionamento del manager 31

4.1 Speedup di Faskel 36
4.2 Efficienza di Faskel al variare della grana e del numero di worker 37
4.3 Aumento di prestazioni dovuto all'utilizzo di funzioni inline . . 38
4.4 Aumento di prestazioni dovuto all'utilizzo del pool di MDFg . 38
4.5 Struttura di ffProbe 41

Elenco dei codici

2.1 Creazione di nodi in FastFlow 12
2.2 Creazione di una farm. 13
3.1 Realizzazione di un task. 16
3.2 Realizzazione degli stream. 17
3.3 Creazione di un grafo macro data flow. 20
3.4 Creazione di un nuovo skeleton e modifica alla routine di compilazione. 21
3.5 Creazione di un computable. 23
3.6 Creazione di una pipeline. 25

Introduzione

Questo tirocinio si è svolto presso il Dipartimento di Informatica dell'Università di Pisa e ha avuto come obiettivo la riscrittura di una libreria per la programmazione a skeleton [1] già esistente sfruttando un diverso tipo di tecnologie.

La libreria in questione è *Muskel* [2, 3], una libreria scritta in Java che fornisce la possibilità di eseguire programmi paralleli su un insieme di macchine connesse da una rete.

Muskel, oltre a fornire alcuni skeleton classici quali *farm* e *pipeline*, dà la possibilità di definire dei grafi che rappresentino delle computazioni parallele: i grafi macro data flow. In questi grafi i nodi rappresentano parti di codice da eseguire sequenzialmente, mentre gli archi corrispondono al flusso dei dati all'interno del grafo. In alternativa, possiamo vedere i nodi come funzioni con un numero arbitrario di argomenti e di valori di ritorno. Se due istruzioni x e y sono collegate da un arco, allora, almeno un valore restituito da x diventerà un parametro di ingresso di y.

Lo scopo del tirocinio era la realizzazione di **Faskel**, una libreria per la programmazione a skeleton con un'interfaccia equivalente a quella di Muskel. A differenza di Muskel, però, bisognava permettere l'esecuzione dei programmi su macchine multicore a memoria condivisa anziché in ambiente distribuito. L'obiettivo iniziale era la realizzazione degli skeleton *farm* e *pipeline* e del supporto per l'esecuzione di grafi macro data flow generici. Inoltre la libreria doveva permettere la composizione arbitraria fra gli skeleton forniti e la compilazione di essi in grafi macro data flow. Tutti gli obiettivi iniziali sono stati raggiunti e successivamente sono stati introdotti anche gli skeleton *map* e *reduce*, con la possibilità di comporli in modo arbitrario.

Uno dei requisiti del progetto era quello di sviluppare la libreria utilizzando il linguaggio di programmazione *C++*, motivo per il quale la prima parte del lavoro è stata dedicata allo studio degli aspetti caratteristici di tale linguaggio.

Per poter eseguire gli skeleton è stata utilizzata la libreria *FastFlow* [4, 5]. L'utilizzo di FastFlow era un requisito iniziale ed essenziale del tirocinio, di

conseguenza una certa quantità di tempo è stata investita nello studio del funzionamento e dell'utilizzo di questa libreria. FastFlow è un framework che fornisce skeleton altamente efficienti su macchine a memoria condivisa. L'efficenza viene raggiunta grazie ad alcuni accorgimenti quali l'utilizzo di code *lock-free* e l'assenza di meccanismi di sincronizzazione. La farm fornita da FastFlow è stata utilizzata all'interno della libreria per realizzare l'interprete dei grafi macro data flow. Le istruzioni dei grafi vengono inviate all'istanza della farm che provvede in seguito a distribuirle ai suoi *workers*. Dato che ogni worker è rappresentato da un thread in esecuzione su un diverso core, questo permette alle istruzioni, e quindi al grafo che rappresenta la computazione, di essere eseguito in parallelo.

Inoltre, per analizzare a fondo le capacità del framework in una situazione reale, è stata sviluppata una cache Netflow [6]. Si tratta fondamentalmente di un analizzatore di traffico di rete che raccoglie informazioni, le memorizza in una cache gestita da più thread ed infine le esporta verso un host remoto. Questa cache originariamente è stata sviluppata per il corso di *Gestione di Reti* utilizzando FastFlow, e in seguito modificata in modo da utilizzare il framework sviluppato durante il tirocinio.

Dopo aver verificato la correttezza del funzionamento, sono stati effettuati test con l'obiettivo di verificare l'efficenza e la scalabilità del framework.

La relazione di tirocinio è strutturata come segue:

- nel primo capitolo approfondiremo il concetto di skeleton e analizzeremo quelli forniti da Faskel;
- nel secondo capitolo analizzeremo gli strumenti utilizzati. Introdurremo le principali caratteristiche di *C++* e mostreremo la struttura di FastFlow, punto centrale nell'implementazione della libreria;
- nel terzo capitolo vedremo la struttura e l'implementazione di Faskel, fornendo anche alcuni esempi di utilizzo;
- nel quarto capitolo analizzeremo i risultati sperimentali ottenuti utilizzando la libreria, facendo le dovute considerazioni sulla scalabilità al variare della grana computazionale;
- infine nel quinto capitolo diamo le conclusioni sul lavoro svolto, analizzandone i limiti e proponendo alcuni possibili sviluppi futuri.

Capitolo 1

Programmazione a skeleton

Uno skeleton è un pattern di programmazione che permette di modellare forme tipiche di esplicitazione di parallelismo espresse in forma parametrica. A partire dal problema sequenziale, è solitamente possibile individuare uno schema predefinito che lo modelli e che riesca quindi ad esprimerlo in forma parallela. Dato che spesso un problema reale può essere modellato in modi diversi, dunque con diversi skeleton, spetta al programmatore individuare quello più adatto ed efficente in una determinata situazione. Una volta individuato lo skeleton opportuno, bisognerà organizzare il programma in parti che corrisponderanno ai diversi parametri dello skeleton.

L'esempio probabilmente più immediato per capire questo concetto è quello della moltiplicazioni di due matrici quadrate, mostrato da Cole in [7]. Consideriamo una matrice C di dimensione m, risultante dal prodotto di due matrici A e B di dimensione m. Suddividendo ciascuna matrice in quattro quadranti abbiamo:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} \tag{1.1}$$

dove $C_{ij} = (A_{i1} \times B_{1j}) + (A_{i2} \times B_{2j})$ con i e $j \in \{1, 2\}$. In questo modo, il prodotto fra le due matrici può essere espresso tramite otto prodotti di matrici con $\frac{m}{4}$ elementi e quattro somme di matrici con $\frac{m}{4}$ elementi. Come possiamo vedere, è il calcolo stesso a suggerire una semplice divisione in parti e una possibile interazione fra esse. Esprimendo il prodotto in questa forma, infatti, si possono eseguire le otto moltiplicazioni fra sottomatrici in parallelo fra di loro. Le quattro somme possono essere eseguite in parallelo (oppure in serie) dopo i prodotti. Nella sezione 3.3.4 viene mostrato uno dei possibili modi per risolvere questo problema utilizzando una *map* (la struttura della *map* viene introdotta nella sezione 1.2.1).

La maggior parte dei sistemi per la programmazione a skeleton esistenti operano in ambiente distribuito. Possiamo suddividere questi sistemi in due categorie, a seconda del modo in cui è possibile definire gli skeleton:

utilizzo di linguaggi per la programmazione parallela In questo caso i sistemi forniscono un linguaggio che permetta di definire degli skeleton tramite costrutti primitivi. Questi linguaggi possono essere:

- estensioni di linguaggi di programmazione già esistenti (Eden [8], PAS, EPAS);
- linguaggi creati *ad-hoc* (ASSIST [9, 10], P^3L [11], Skie [12]).

utilizzo di una libreria Le librerie sono costruite per un determinato linguaggio di programmazione e mettono a disposizione funzioni per la creazione di skeleton. Oltre a Muskel e FastFlow (che non opera in ambiente distribuito), possiamo ricordare eSkel [13, 14], SkeTo [15] e Muesli [16].

Come specificato da Cole nel suo manifesto [13], la programmazione a skeleton offre diversi vantaggi:

- semplifica la programmazione tramite l'astrazione di alcuni meccanismi (sincronizzazione e interazione fra thread, ecc...), diminuendo quindi la possibilità di errori dovuti a deadlock e inconsistenze;
- garantisce la portabilità e il riuso, sollevando il programmatore dal compito di implementare e definire i pattern sottostanti;
- aumenta la performance, fornendo per ogni pattern un'implementazione ottimizzata per un determinato tipo di architettura;
- ottimizza il codice, grazie alla conoscenza a priori dei pattern utilizzati.

Un linguaggio basato su skeleton (o una libreria per la programmazione a skeleton), deve quindi fornire al programmatore una serie di skeleton e possibilmente dei meccanismi per combinarli fra di loro. In quest'ultimo caso i parametri dello skeleton anziché essere delle funzioni, potranno essere a loro volta degli skeleton.

E' possibile suddividere gli skeleton in due gruppi principali, a seconda del tipo di parallelismo che esprimono, come mostrato in [17]:

- *stream parallel.* Il parallelismo è raggiunto grazie all'applicazione della funzione f, calcolata dallo skeleton, su diversi elementi dello stream di ingresso. Per questo motivo, gli skeleton stream parallel hanno senso solo se vengono eseguiti su un flusso di dati;

- *data parallel.* Il parallelismo è raggiunto grazie all'applicazione della funzione f, calcolata dallo skeleton, su parti diverse dello stesso elemento in ingresso. Questi skeleton quindi possono essere utilizzati anche in assenza di un flusso di dati.

Di seguito vengono illustrati brevemente alcuni degli skeleton più comuni, forniti anche da Faskel.

1.1 Skeletons stream parallel

1.1.1 Farm

Siano dati una funzione $f : T \longrightarrow V$ e uno stream di elementi $\{x_1, x_2, \ldots, x_n\}$ di tipo T. La farm è uno skeleton che calcola

$$y_i = f(x_i) \quad \forall i : 1 \leq i \leq n$$

e restituisce gli elementi $y_1, y_2, \ldots, y_n$ di tipo V.

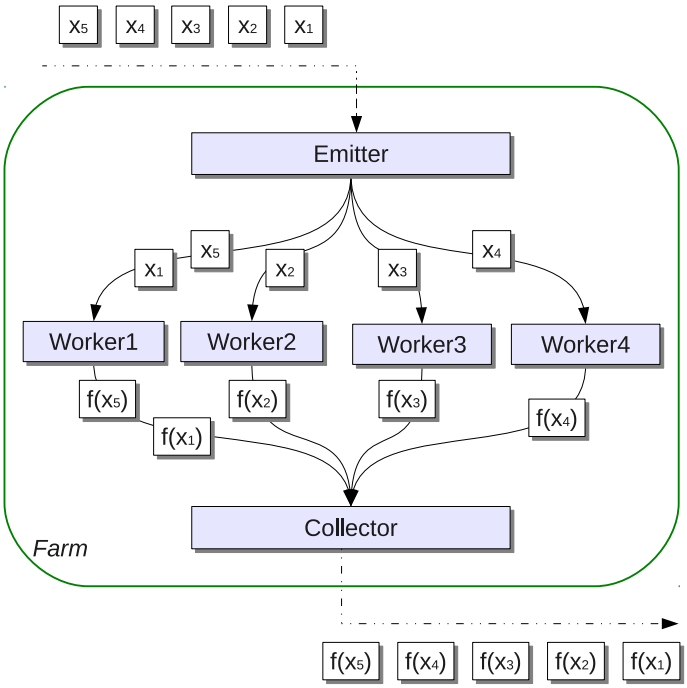

Figura 1.1: Farm con 4 workers

Una farm è implementata solitamente da un *emettitore*, da un certo numero di *worker* e da un *collettore*, come mostrato in figura 1.1. L'emettitore

assegna ogni elemento in ingresso ad un determinato worker. In questo modo più worker contemporaneamente possono essere in esecuzione, diminuendo quindi il tempo necessario al calcolo di f su tutti gli elementi. Infine il collettore riceve i risultati calcolati e li restituisce nell'ordine corretto.

Il vantaggio nell'utilizzare una farm è che calcoli su elementi successivi dello stream possono essere eseguiti in parallelo da thread diversi.

1.1.2 Pipeline

Siano date le funzioni $\{f_1 : T_0 \longrightarrow T_1, f_2 : T_1 \longrightarrow T_2, \ldots, f_n : T_{n-1} \longrightarrow T_n\}$ e uno stream di elementi $\{x_1, x_2, \ldots, x_m\}$ di tipo T_0. La pipeline è uno skeleton che calcola

$$y_i = f_n(f_{n-1}(\ldots(f_1(x_i))\ldots)) \quad \forall i : 1 \leq i \leq m$$

e restituisce gli elementi $y_1, y_2, \ldots, y_m$ di tipo T_n.

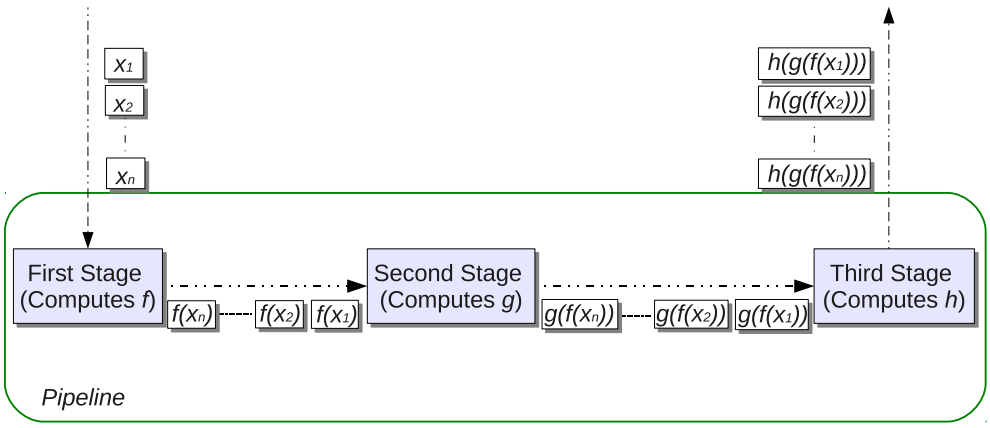

Figura 1.2: Pipeline a 3 stadi

La pipeline è implementata da una serie di *stadi*, solitamente eseguiti in parallelo. Lo stadio i-esimo riceve in ingresso elementi di tipo T_{i-1}, calcola su di essi la funzione f_i e invia elementi di tipo T_i allo stadio successivo.

Il vantaggio di strutturare questo calcolo come una pipeline consiste nel poter calcolare stadi diversi in parallelo.

Supponiamo ad esempio di avere una pipeline come quella in figura 1.2. Sia $y_i = f(x_i)$ il valore calcolato dal primo stadio e $z_i = g(y_i)$ il valore calcolato dal secondo stadio. In questo caso, mentre il terzo thread esegue $h(z_i)$, il secondo thread calcola $g(y_{i+1})$ e il primo thread può calcolare $f(x_{i+2})$.

1.2 Skeletons data parallel

1.2.1 Map

Siano dati una funzione $f : T \longrightarrow V$ e uno stream di vettori di dimensione variabile $\{X_1 = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1k} \end{bmatrix}, X_2 = \begin{bmatrix} x_{21} & x_{22} & \cdots & x_{2t} \end{bmatrix}, \ldots, X_n = \begin{bmatrix} x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}\}$ con componenti di tipo T. La map è uno skeleton che calcola

$$Y_i = f(X_i) = \begin{bmatrix} f(x_{i1}) & f(x_{i2}) & \cdots & f(x_{ir}) \end{bmatrix} \quad \forall i : 1 \leq i \leq n$$

e restituisce i vettori $Y_1, Y_2, \ldots, Y_n$ con componenti di tipo V. Il concetto

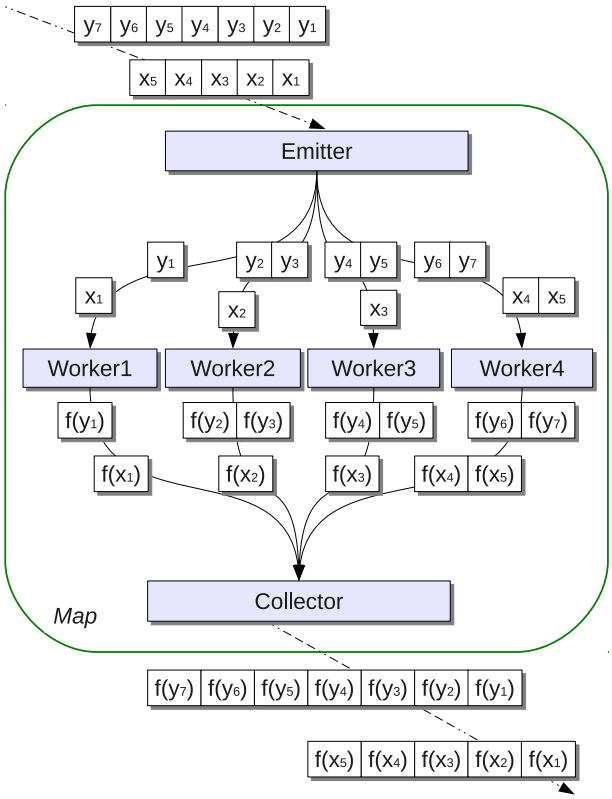

Figura 1.3: Map con 4 workers

di vettore in questo caso può essere considerato in un accezione più ampia, fino ad includere un qualsiasi tipo di collezione, senza alterare il tipo di parallelismo espresso.

Una map è implementata solitamente da un *emettitore*, da w *worker* e da un *collettore*, come mostrato in figura 1.3.

L'emettitore si occupa di dividere i vettori in ingresso in w sottovettori di dimensione più piccola e di passare questi sottovettori ai worker.

Ogni worker riceve un sottovettore dall'emettitore e calcola $f(x_{ij})$ su ogni componente x_j del sottovettore ricavato dal vettore X_i restituendo quindi un sottovettore con componenti di tipo V.

Infine il collettore riceve w sottovettori dai worker e li unisce in modo da formare un vettore con componenti di tipo V, ordinato in modo coerente col vettore iniziale.

Il vantaggio rispetto alla versione sequenziale è evidente e consiste nel fatto che la funzione f può essere calcolata contemporaneamente da w thread diversi su diverse componenti di uno stesso vettore.

1.2.2 Reduce

Siano dati una funzione $f : T \longrightarrow T$ binaria, commutativa e associativa e uno stream di vettori di dimensione variabile $\{X_1 = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1k} \end{bmatrix}, X_2 = \begin{bmatrix} x_{21} & x_{22} & \cdots & x_{2t} \end{bmatrix}, \ldots, X_n = \begin{bmatrix} x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}\}$ con componenti di tipo T. La reduce è uno skeleton che calcola

$$y_i = \underbrace{f(x_1, \underbrace{f(x_2, \underbrace{f\,(x_3, f(\ldots, x_n)\ldots)}))}}} \quad \forall i : 1 \leq i \leq n$$

e restituisce gli elementi $y_1, y_2, \ldots, y_n$ di tipo T.

Anche in questo caso al posto dei vettori è possibile utilizzare qualunque collezione. La proprietà associativa della funzione utilizzata permette di utilizzare uno schema di parallelizzazione. Una reduce è implementata solitamente da un *emettitore*, da w *worker* e da un *collettore*, come mostrato in figura 1.4.

Sia k la dimensione di un generico vettore in ingresso. L'emettitore si occupa di dividere i vettori in w sottovettori di dimensione $\frac{k}{w}$ e di passare questi sottovettori ai worker.

Ogni worker riceve un sottovettore dall'emettitore e calcola la riduzione sulle componenti del sottovettore. Ovviamente in molti casi avremo $k > n$ e di conseguenza il sottovettore ricevuto avrà più di due componenti; motivo per cui alcune riduzioni andranno calcolate in modo sequenziale all'interno dei worker.

Infine il collettore riceve le w riduzioni dai worker e calcola a sua volta la riduzione sugli elementi ricevuti.

Il vantaggio rispetto alla versione sequenziale è evidente e consiste nel fatto che la funzione f può essere calcolata contemporaneamente da w thread diversi su diverse coppie di componenti di uno stesso vettore.

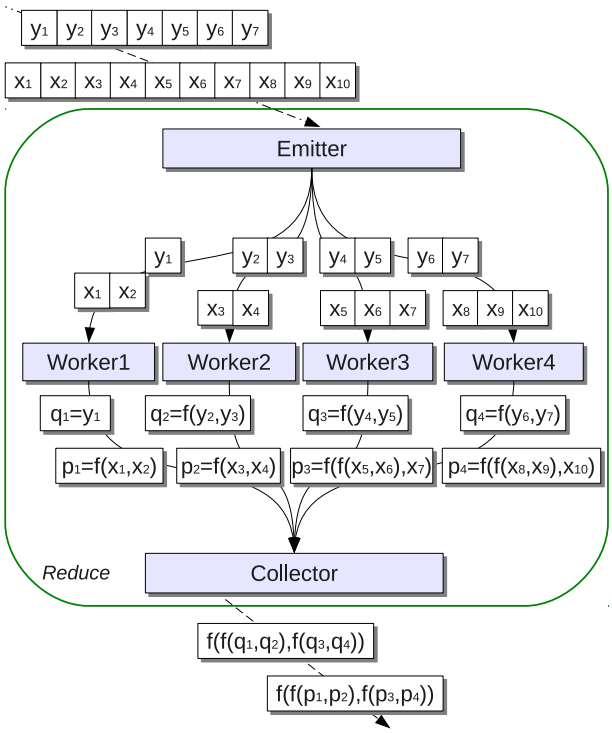

Figura 1.4: Reduce con 4 workers

1.3 Sequenziale

Lo skeleton sequenziale ha soltanto lo scopo di incapsulare una funzione che non si può o che non si vuole parallelizzare ulteriomente. Come accennato in precedenza gli skeletons possono essere definiti in modo che siano composti a loro volta da altri skeletons; per questo motivo anche le parti che vanno eseguite in modo sequenziale devono essere degli skeletons.

Se consideriamo una composizione fra skeletons e la rappresentiamo sotto forma di albero, gli skeleton sequenziali andranno sempre a formare le foglie dell'albero, come mostrato in figura 1.5.

In Faskel lo skeleton sequenziale non può essere dichiarato in modo esplicito poiché è stata adottata una soluzione alternativa ed equivalente, mostrata nella sezione 3.3.1.

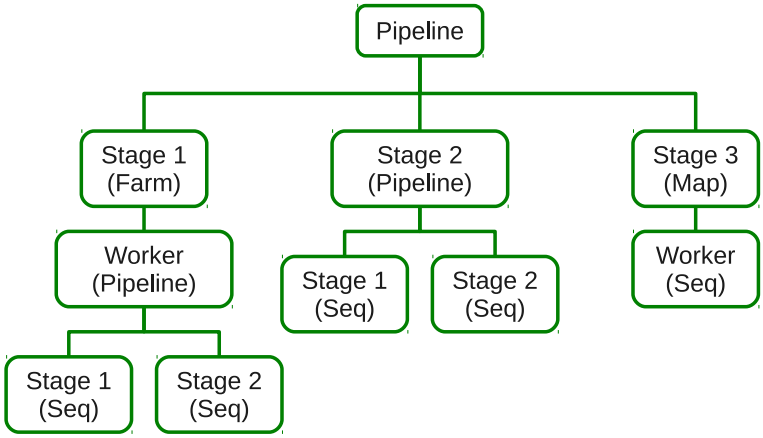

Figura 1.5: Composizione di skeleton

Capitolo 2

Strumenti utilizzati

In questo capitolo illustreremo brevemente gli strumenti utilizzati per la realizzazione della libreria. L'utilizzo di questi strumenti era un requisito iniziale e la prima parte del tirocinio è stata dedicata allo studio e all'utilizzo di essi.

2.1 C++

Il *C++* [18] è un linguaggio di programmazione multi-paradigma con tipizzazione statica, sviluppato da Bjarne Stroustrup a partire dal 1979 presso i Bell Labs. L'obiettivo di Stroustrup era quello di creare un linguaggio che mantenesse le caratteristiche del *C* [19], cioè efficenza, portabilità e meccanismi di basso livello, ma che allo stesso tempo fornisse alcuni meccanismi ad alto livello come la programmazione ad oggetti.

Il *C++* viene definito multi-paradigma poiché mette a disposizione sia la programmazione imperativa, tipica del *C*, sia la programmazione ad oggetti e quella generica. Si distingue inoltre dal *C* per altre caratteristiche come:

- funzioni inline: sono funzioni vere e proprie ma vengono trattate dal compilatore in modo diverso rispetto alle funzioni normali. Il compilatore, infatti, sostituisce tutte le chiamate a queste funzioni con il codice della funzione stessa. Questa soluzione ha il vantaggio di eliminare il costo dell'inserimento dei parametri sullo stack e di migliorare la località dei riferimenti. Rispetto alle macro, le funzioni inline sono più sicure poiché effettuano il controllo sui tipi. Tuttavia è consigliabile utilizzare la direttiva `inline` soltanto per funzioni composte da poche righe di codice, poiché altrimenti aumenta la dimensione del programma e di conseguenza peggiora la località dei riferimenti. Nello sviluppo della libreria le funzioni inline sono state utilizzate in modo sostanziale;

- funzioni virtuali: sono alla base del binding dinamico e possono essere definite ed implementate all'interno di una classe. Se le classi derivate ridefiniscono una funzione virtuale dichiarata all'interno della classe che estendono, allora a tempo di esecuzione verrà eseguita la funzione corretta a seconda dell'oggetto su cui si sta invocando la funzione;

- sovraccaricamento delle funzioni: permette di utilizzare lo stesso nome per funzioni diverse all'interno dello stesso programma, a patto che le funzioni abbiano tipi diversi per i parametri utilizzati. A tempo di esecuzione verrà eseguita la funzione appropriata, grazie alla conoscenza del tipo degli argomenti passati;

- template: è possibile definire una funzione (o una classe) template inserendo l'espressione `template<class T>` prima della dichiarazione.

  - funzioni template: sono funzioni che possono accettare o restituire valori di tipo `T`. Grazie a questo meccanismo è quindi possibile definire funzioni parametriche rispetto ad un tipo;

  - classi template: analogamente alle funzioni template, queste classi possono essere parametriche rispetto al tipo `T` specificato durante la dichiarazione;

  I parametri di un template possono anche essere più di uno. In tal caso, tutti i tipi vanno specificati nella definizione del template. Ad esempio: `template<class T1, class T2, ..., class Tn>`. I template sono stati utilizzati nella libreria per permettere la creazione di skeleton parametrici rispetto ai tipi specificati dal programmatore;

- riferimenti: sono degli "alias" di altre variabili e vanno inizializzato nello stesso momento in cui sono dichiarati. Qualsiasi modifica apportata al riferimento si riflette sul valore riferito e viceversa. L'utilizzo tipico si ha nel passaggio degli argomenti alle funzioni; grazie a questo meccanismo infatti è possibile realizzare il passaggio di parametri per riferimento. Il vantaggio rispetto al passaggio per valore è evidente per parametri di grosse dimensioni poiché si evita la copia sullo stack;

- altre caratteristiche come eccezioni, ereditarietà multipla, parametri di default, namespace e costanti sono ampliamente trattate in [18].

2.2 FastFlow

FastFlow è un framework scritto in *C++* per la programmazione a skeleton su piattaforme multicore a memoria condivisa. Questo framework è strutturato a livelli che astraggono man mano l'architettura sottostante, ed offre una serie di meccanismi di comunicazione a bassa latenza sfruttando delle code *lock-free*.

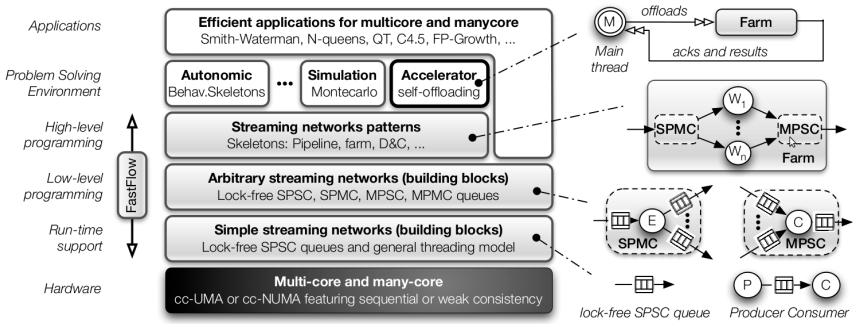

Figura 2.1: Architettura di FastFlow

Il processo di astrazione effettuato dai livelli di FastFlow (figura 2.1) ha un duplice obiettivo. Da un lato facilitare la programmazione fornendo costrutti ad alto livello per la creazione di skeleton *stream parallel*; dall'altro garantire l'efficenza mettendo a disposizione un insieme di meccanismi di basso livello. Fra questi meccanismi possiamo trovare delle code utilizzabili da un singolo produttore e da un singolo consumatore (d'ora in avanti chiamate code *SPSC*). Queste code sono caratterizzate dal fatto di non aver bisogno di meccanismi di *lock* e sono state realizzate traendo ispirazione dai protocolli *wait-free* descritti da Lamport in [20] e dalle code di FastForward descritte in [21].

Purtroppo la soluzione adottata presenta alcuni svantaggi: innanzitutto i thread che condividono la coda, per sincronizzarsi, hanno bisogno di effettuare attesa attiva, sprecando quindi risorse. Inoltre, come vedremo in seguito, le code con più consumatori o più produttori, richiedono un thread addizionale per essere utilizzate. Nonostante ciò, come mostrato in [22], è preferibile non avere meccanismi di *lock*. Questo è evidente soprattutto quando la grana della computazione[1] è fine, poiché i meccanismi di sincronizzazione

[1]La grana della computazione è una quantità definita come $\frac{T_{ex}}{T_{com}}$, dove T_{ex} è il tempo impiegato dal worker per eseguire il task, mentre T_{com} è il tempo impiegato per trasmettere un task al worker e per riceverne il risultato.

introducono un overhead notevole a causa delle continue invalidazioni della cache.

Queste code possono essere utilizzate importando il file *ff/buffer.hpp* ed invocando il costruttore `SWSR_Ptr_Buffer(size_t n)`, dove `n` rappresenta la dimensione massima della coda. Una volta creata, è possibile invocare su di essa vari metodi, tra cui i classici `push` che inserisce un elemento in fondo alla coda, e `pop` che estrae un elemento dalla testa. Esiste anche un estensione di queste code, definita in *ff/ubuffer.hpp*, senza alcun limite sul numero massimo di elementi che possono contenere.

FastFlow, inoltre, dà la possibilità di definire dei nodi che rappresentino computazioni da eseguire in thread indipendenti. Per utilizzarli bisogna importare il file *ff/node.hpp*, creare una sottoclasse di `ff::ff_node` e specificare il codice da eseguire implementando la funzione virtuale `svc`, come mostrato nell'esempio 2.1.

Codice 2.1: Creazione di nodi in FastFlow

```
1  #include <ff/node.hpp>
2  using namespace ff;
3
4  class Emitter: public ff_node {
5  public:
6      Emitter(int max_task):ntask(max_task) {};
7
8      void * svc(void *) {
9          int * task = new int(ntask);
10         --ntask;
11         if (ntask<0) return NULL; //End of stream
12         return task;
13     }
14 private:
15     int ntask;
16 };
17
18 class Worker: public ff_node {
19 public:
20     void * svc(void * task) {
21         int * t = (int *)task;
22         (*t)++;
23         return task;
24     }
25 };
26
27 class Collector: public ff_node {
```

```
28 public:
29     void * svc(void * task) {
30         int * t = (int *)task;
31         if (*t == -1) return NULL;
32         return task;
33     }
34 };
```

Una volta creati i nodi e le code SPSC, è possibile creare dei semplici grafi computazionali come quelli mostrati in figura 2.2. Per assegnare ad un nodo un buffer in entrata o in uscita, bisogna invocare su di esso i metodi `set_input_buffer(FFBUFFER * const i)` o `set_output_buffer(FFBUFFER * const o)`, dove `FFBUFFER` è un buffer di dimensione limitata (se è stata specificata la macro `FF_BOUNDED_BUFFER`) o illimitata. Ovviamente ciascun nodo ha un ruolo prefissato rispetto ad ognuna delle code che condivide: o si comporta da produttore o si comporta da consumatore.

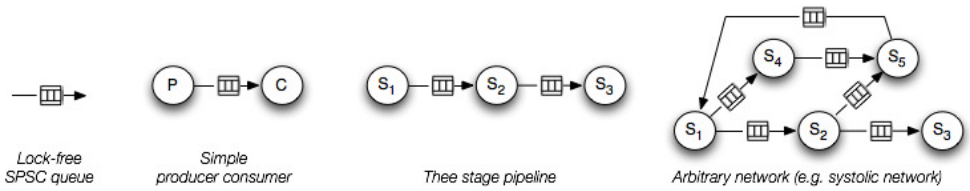

Figura 2.2: Esempi di grafi definibili utilizzando le code SPSC.

FastFlow fornisce anche alcuni skeleton *stream parallel* come farm, farm con feedback e pipeline. La farm con feedback è un estensione della farm in cui alcuni risultati calcolati dal collettore possono essere inviati all'emettitore. Inoltre l'insieme degli skeletons può essere facilmente esteso creando composizioni arbitrarie fra essi o definendone di nuovi.

L'esempio 2.2 mostra le operazioni da eseguire per creare una farm utilizzando i nodi definiti nell'esempio 2.1. Innanzitutto bisogna includere l'header in cui la farm è definita, cioè *ff/farm.hpp*. Dopodiché si può creare un'istanza di `ff::ff_farm` (linea 6), aggiungere ad essa l'emettitore, i worker e il collettore (linee 12-14) ed infine avviare l'esecuzione (linea 15). L'utilizzo del collettore e dell'emettitore non è obbligatorio. Se il collettore non è presente, allora i worker dovranno restituire sempre il valore `GO_ON`.

Codice 2.2: Creazione di una farm.

```
1 #include <ff/farm.hpp>
2 using namespace ff;
```

```
3  #define NWORKERS 4
4  int main(){
5       std::vector<ff_node *> w;
6       ff_farm<> farm;
7       Emitter e;
8       Collector c;
9       for(int i=0; i<NWORKERS; ++i){
10           w.push_back(new Worker);
11      }
12      farm.add_emitter(&e);
13      farm.add_workers(w);
14      farm.add_collector(&c);
15      farm.run();
16      return 1;
17 }
```

FastFlow inoltre mette a disposizione un acceleratore software, cioè un insieme di meccanismi per trasformare in modo semi-automatico un programma C/C++ sequenziale in un'equivalente versione parallela [23]. Un acceleratore incapsula uno skeleton (una farm nel caso di FastFlow) e fornisce al programmatore un canale di ingresso e uno di uscita su cui poter inviare i task e ricevere i risultati. Una volta specificato il codice da far eseguire ai worker della farm, i task possono essere inviati all'acceleratore utilizzando il metodo `offload`. Se necessario, i risultati possono essere ottenuti dal canale di uscita invocando i metodi `load_result` e `load_result_nb`. L'acceleratore, come verrà mostrato nella sezione 3.5, viene utilizzato all'interno della libreria per realizzare l'interprete dei grafi macro data flow.

Infine, bisogna fare una considerazione riguardo l'impatto che ha sulle prestazioni l'utilizzo di un allocatore di memoria dinamica. L'allocatore standard di *C++*, se viene utilizzato contemporaneamente da più thread, adopera dei meccanismi di mutua esclusione per garantire la consistenza delle sue strutture dati, reintroducendo quindi i problemi relativi ai *lock*. Per questo motivo FastFlow mette a disposizione un allocatore di memoria dinamica che non utilizza meccanismi di mutua esclusione. Questo allocatore può essere impiegato da un solo thread per allocare memoria, mentre non esiste limite sul numero di thread che possono effettuare deallocazioni. FastFlow fornisce anche un altra versione dell'allocatore, senza alcun limite sul numero di thread allocatori, ma con prestazioni non altrettanto buone. L'allocatore di FastFlow, nella versione con un sono allocatore, è stato utilizzato nella realizzazione della cache NetFlow (sezione 4.4).

Capitolo 3

Faskel: struttura ed implementazione

L'implementazione del framework ha costituito la parte fondamentale del tirocinio. La libreria è suddivisa fra vari file **.hpp*, da includere a seconda delle esigenze.

Per utilizzare la libreria bisogna compiere le seguenti operazioni:

- includere il file *./Muskel/Manager.hpp*;
- realizzare i task e gli stream;
- individuare uno skeleton o un grafo macro data flow che modelli il problema e crearne un'istanza;
- creare un'istanza del manager passandogli i parametri opportuni ed invocare su di essa il metodo `exec`.

Di seguito verranno approfondite le operazioni da compiere, saranno illustrati gli elementi fondamentali della libreria e verranno mostrati alcuni esempi di utilizzo. Inoltre, verranno trattate le principali scelte implementative effettuate evidenziando i lati positivi e quelli negativi di tali scelte.

3.1 Task e streams

Una delle attività fondamentali durante la costruzione di un programma strutturato a skeleton è la definizione dei *task*. Uno skeleton infatti riceve uno stream di task rappresentante i dati in ingresso e restituisce uno stream di task che rappresentano i risultati calcolati.

Per definire un task bisogna creare una classe che sia sottoclasse della classe `Task`. Questa classe è una *marker interface* definita nel file *Task.hpp*. L'esempio 3.1 mostra l'implementazione di un semplice task che rappresenta un intero.

Codice 3.1: Realizzazione di un task.

```
1 #include <Task.hpp>
2 class IntTask: public Task{
3 private:
4      int x;
5 public:
6      IntTask(int x):this.x(x){;}
7      inline int get(){return x;}
8      inline void set(int t){x=t;}
9 }
```

Per quanto riguarda la creazione degli stream, è stata effettuata una scelta che permette al programmatore di incapsulare qualsiasi tipo di flusso. Per creare uno stream è necessario estendere la classe `InputStream` o `OutputStream`, definite in *Muskel/Streams.hpp* ed implementarne i metodi. In questo modo lo stream può rappresentare un file, lo standard input, la rete o altro. Questa è la stessa soluzione adottata da Muskel, eccezion fatta per un aspetto riguardante la gestione dello stream di ingresso. In Muskel infatti lo stream di ingresso è bloccante e viene gestito da un thread separato, permettendo quindi al manager di continuare a gestire le MDFi anche nel caso in cui non siano presenti task sullo stream. In Faskel questo problema è stato affrontato ma, per ragioni di tempo, la soluzione non è stata realizzata. Per questo motivo bisogna gestire gli input stream in modo non bloccante, cioè in modo che restituiscano `NULL` se momentaneamente non è disponibile nessun nuovo valore. In caso contrario il manager resterà bloccato fino al momento in cui non avrà letto `groupSize` task (il significato di `groupSize` è definito nella sezione 3.6).

L'input stream deve fornire i metodi:

- `Task* next()` che rappresenta la lettura dei dati dallo stream e restituisce un puntatore ad un task. Questo metodo restituisce `NULL` nel caso in cui al momento non siano disponibili altri task;

- `bool hasNext()` che restituisce *false* nel caso in cui sia arrivato l'*end-of-stream* e quindi non bisogna attendere altri dati.

L'output stream invece deve fornire il metodo:

- `void put(Task* a)` che gestisce il task calcolato. Questo metodo corrisponde all'invio del task sullo stream di output.

L'esempio 3.2, sfruttando il task definito nell'esempio 3.1, mostra un implementazione di un input stream che legge dalla rete elementi che rappresentano interi e di un output stream che stampa su un file i risultati calcolati.

Codice 3.2: Realizzazione degli stream.

```
1  #include <Muskel/Streams.hpp>
2
3  class MyInputStream: public muskel::InputStream{
4  private:
5       int i, elements;
6  public:
7       MyInputStream(int elements):i(0),this.elements(elements){;}
8
9       Task* next(){
10           i++;
11           int x;
12           NetworkPacket* p;
13           p=readDataFromNetwork();
14           //Parses the packet.
15           x=PacketToInt(p);
16           IntTask* toReturn=new IntTask(x);
17           return toReturn;
18      }
19
20      inline bool hasNext(){
21           return i<elements;
22      }
23 }
24
25 class MyOutputStream: public muskel::OutputStream{
26 public:
27      void put(Task* a){
28           //To note the explicit cast from Task*
29           //to IntTask*.
30           IntTask* x=(IntTask*) a;
31           //Prints the integer on file.
32           printOnFile(x);
33           delete x;
34      }
35 }
```

3.2 Grafi macro data flow

I grafi macro data flow (*MDFg* dall'inglese *Macro Data Flow graphs*) sono dei grafi tramite i quali è possibile esprimere computazioni parallele arbitrarie. In questi grafi i nodi rappresentano parti di codice da eseguire in modo sequenziale, mentre gli archi rappresentano il flusso dei dati calcolati all'interno del grafo. I nodi vengono anche chiamati "istruzioni macro data flow" (*MDFi* dall'inglese *Macro Data Flow Instructions*), con il termine *macro* che sta ad indicare il fatto che una singola istruzione rappresenta in realtà un calcolo consistente.

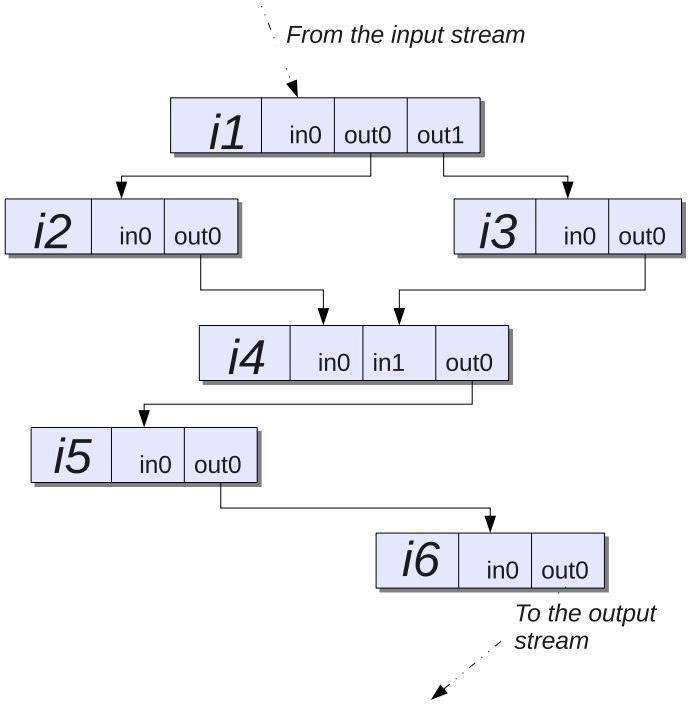

Figura 3.1: Esempio di un grafo macro data flow

È importante capire la relazione che è presente fra elementi dello stream e MDFg. Per ogni task ricevuto dallo stream di ingresso, infatti, viene istanziata una copia del grafo che viene poi distrutta alla fine del calcolo. Ogni istanza del grafo, quindi, si occupa di eseguire il calcolo soltanto su un elemento dello stream di ingresso, motivo per il quale i grafi macro data flow

hanno senso soltanto su applicazioni che lavorano su stream di dati. Una MDFi, come spiegato in [2], può essere vista come una quintupla

$$MDFi \equiv Id \times Id \times Code \times I^n \times O^m$$

dove il primo valore rappresenta l'identificatore utilizzato per distinguere l'istruzione dalla altre istruzioni del grafo, il secondo valore identifica l'istanza del grafo a cui appartiene l'istruzione e il terzo valore rappresenta un puntatore al codice che la MDFi deve eseguire. Infine, il quarto e il quinto valore rappresentano rispettivamente gli n token di input e le m destinazioni dei token di output.

I token rappresentano i valori ricevuti o calcolati da una MDFi. Si dice che una MDFi diventa *fireable* nel momento in cui riceve tutti i token di ingresso. Ad esempio, nel grafo in figura 3.1, l'istruzione `i4` diventa *fireable* solo dopo aver ricevuto i risultati dalle MDFi `i2` e `i3`. Soltanto a questo punto è possibile eseguire l'istruzione e inviare i token che rappresentano i risultati alle MDFi collegate ad essa da archi. Non esistono vincoli sul numero di token che una MDFi può ricevere o produrre, eccezion fatta per la prima e l'ultima istruzione del grafo. La prima, infatti, deve avere soltanto un token in ingresso, corrispondente al valore ricevuto dallo stream; l'ultima deve avere un solo valore in uscita, destinato allo stream di output. Questo vincolo è presente per permettere ai grafi di essere annidati in modo arbitrario fra di loro.

Nella libreria i MDFg sono definiti nel file *Muskel/MDF/Mdfg.hpp*. Per utilizzarli bisogna creare un istanza `g` della classe `mdf::Mdfg` ed invocare su di essa una serie di metodi:

1. `mdf::Mdfi* setFirstMdfi(skel::Computable, int)`: crea la prima istruzione del grafo. Il primo parametro rappresenta la funzione da calcolare nella prima istruzione e il secondo indica il numero di output dell'istruzione (il numero di input è implicitamente uno e corrisponde allo stream di ingresso);

2. `mdf::Mdfi* setMdfi(skel::Computable, int, int)`: crea una generica istruzione del grafo. Il primo parametro rappresenta la funzione da calcolare, il secondo e il terzo indicano rispettivamente il numero di input e di output dell'istruzione;

3. `mdf::Mdfi* setLastMdfi(skel::Computable, int)`: crea l'ultima istruzione del grafo. Il primo parametro rappresenta la funzione da calcolare nella prima istruzione e il secondo indica il numero di input dell'istruzione (il numero di output è esplicitamente uno e corrisponde allo stream di uscita);

4. `bool link(mdf::Mdfi* x, int y, mdf::Mdfi* z, int w)`: collega l' `y`-esimo output dell'istruzione `x` allo `w`-esimo input dell'istruzione `z`. Il metodo restituisce `true` se il collegamento è andato a buon fine.

I metodi utilizzati per creare le istruzioni restituiscono un puntatore all'istruzione creata. L'esempio 3.3 mostra il codice utilizzato per generare il grafo in figura 3.1.

Codice 3.3: Creazione di un grafo macro data flow.

```
1  using namespace mdf;
2  Mdfg* graph = new Mdfg();
3  Mdfi *i1,*i2,*i3,*i4,*i5,*i6;
4
5  /**Creates the first instruction
6  that compute the sequential skeleton
7  seq1. The instruction have two output.
8  (The number of input is implicit one)**/
9  i1=graph->createFirstMdfi(seq1,2);
10
11 i2=graph->createMdfi(seq2,1,1);
12 i3=graph->createMdfi(seq3,1,1);
13 i4=graph->createMdfi(seq4,2,1);
14 i5=graph->createMdfi(seq5,1,1);
15
16 /**Creates the last instruction.
17 The instruction have one input.
18 (The number of output is implicit one)**/
19 i6=graph->createLastMdfi(seq6,1);
20
21 /**Links the instructions together.**/
22 graph->link(i1,0,i2,0);
23 graph->link(i1,1,i3,0);
24 graph->link(i2,0,i4,0);
25 graph->link(i3,0,i4,1);
26 graph->link(i4,0,i5,0);
27 graph->link(i5,0,i6,0);
```

3.3 Implementazione ed utilizzo degli skeleton

La libreria mette a disposizione diversi tipi di skeleton e diversi procedimenti per utilizzarli. Per non appesantire troppo l'esposizione, per ogni skeleton

presentato verranno mostrati solo alcuni dei possibili metodi per crearlo. I metodi mostrati per un determinato skeleton sono validi anche per tutti gli altri, a condizione che si utilizzi la funzione opportuna.

Oltri agli skeleton forniti, è possibile aggiungere altri skeleton alla libreria. Per farlo bisogna definire una classe che estenda la classe `skel::Computable` e che ne implementi il metodo `compute`. Il codice all'interno del metodo compute corrisponde all'esecuzione sequenziale dello skeleton. Se necessario bisogna fornire alcune funzioni che verranno poi eventualmente utilizzate dal compilatore. Queste funzioni non sono standard ma vanno definite dall'utente a seconda del modo in cui si vuole compilare il grafo (ad esempio funzioni per estrarre i parametri dello skeleton). Per capire meglio come compilare uno skeleton si può approfondire la sezione 3.4, nella quale viene spiegato il modo in cui vengono compilati gli skeleton forniti da Faskel. Inoltre è necessario istruire il compilatore affinché possa trasformare lo skeleton in un grafo macro data flow equivalente. Per fare ciò bisogna aggiungere un nuovo ramo all'`if-then-else` all'interno della routine di compilazione. L'esempio 3.4 mostra la creazione di un nuovo skeleton (linee 3-20) e la conseguente modifica al compilatore (linee 36-40). In alternativa si può definire manualmente il grafo macro data flow corrispondente allo skeleton senza preoccuparsi quindi del processo di compilazione, come mostrato precedentemente nell'esempio 3.3.

Codice 3.4: Creazione di un nuovo skeleton e modifica alla routine di compilazione.

```
1  //FILE: newSkeleton.hpp
2
3  using namespace mdf;
4  namespace skel{
5  class newSkeleton: public skel::Computable{
6  private:
7       skel::Computable* param1;
8       skel::Computable* param2;
9       skel::Computable* param3;
10 public:
11      newSkeleton(skel::Computable* p1, skel::Computable* p2, skel::
       Computable* p3):param1(p1),param2(p2),param3(p3){;}
12
13      Task** compute(Task** t){
14           //Sequential behaviour
15      }
16      skel::Computable* getP1(){return param1;}
17      skel::Computable* getP2(){return param2;}
```

```
18      skel::Computable* getP3(){return param3;}
19 }
20 }
21 ----------------------------------------------------------------------
22 //FILE: Muskel/MDF/Compiler.hpp
23
24 Mdfg* compile(skel::Computable* c){
25      const std::type_info& tc = typeid(*c);
26
27      if(tc==typeid(skel::Farm)){
28           //Farm compiling. Creates a graph
29           //containing the instructions of the worker.
30      }else if(tc==typeid(skel::Pipeline)){
31           //Pipeline compiling. Adds all istructions
32           //of the second stage in the first stage
33           //of the pipeline and links it together.
34      }else if(tc==typeid(skel::EmitterWorkerCollector)){
35           //Map and reduce compiling.
36      }else if(tc==typeid(skel::newSkeleton)){
37           //newSkeleton compiling. Uses getP1, getP2
38           //and getP3 to get the parameters. After that
39           //gets the MDFi, links them together, ecc...
40      }else{
41           //If c is an unknown skeleton,
42           //then is a sequential skeleton.
43           return new Mdfg(c);
44      }
45 }
```

Di seguito verranno mostrati gli skeleton forniti da Faskel ed alcuni esempi di creazione.

3.3.1 Computable

La classe `skel::Computable` rappresenta una computazione generica. Se una classe estende `skel::Computable`, il suo metodo `compute` può essere calcolato in modo sequenziale all'interno di una MDFi. La funzione `Task** compute(Task** t)` riceve in ingresso un vettore di `Task*` e restituisce un vettore di `Task*`, il quale deve essere allocato dinamicamente utilizzando la primitiva `new[]`. Il motivo di questo vincolo è dovuto al fatto che questo vettore viene deallocato in modo automatico dal supporto una volta che questa struttura dati non è più utile. Poiché la deallocazione viene effettuata

tramite l'operatore `delete[]`, l'allocazione va fatta utilizzando la primitiva `new[]`.

L'esempio 3.5 mostra una funzione calcolabile all'interno di una MDFi con n token in ingresso e n token in uscita, utilizzando i task definiti nell'esempio 3.1.

Codice 3.5: Creazione di un computable.

```
1  #include <Skeleton/Computable.hpp>
2
3  class myCompute: public skel::Computable{
4  public:
5       Task** compute(Task** input){
6            IntTask* x;
7            /**Creation of the array to return.
8            (To note that the array is created using new[].**/
9            Task** toReturn=new Task*[n];
10           for(int i=0; i<n; i++){
11                /**Gets the input task (to note the cast).**/
12                x=(IntTask*) input[i];
13                /**Insertion of the elements to return.
14                toReturn[i] will be sent to the MDFi linked
15                to the i-th output.**/
16                toReturn[i]=new IntArrayTask(x[i]->get()+10);
17           }
18      return toReturn;
19      }
20 }
```

Il motivo per cui nella libreria non sia presente lo skeleton sequenziale è dovuto alla presenza di questa classe. Durante lo sviluppo della libreria infatti si è scelto di dare la possibilità al programmatore di esprimere computazioni sequenziali semplicemente estendendo la classe `skel::Computable` e implementandone il metodo `compute`. La ragione dietro questa scelta non è legata a questioni di efficenza o di possibilità implementative ma soltanto a motivi soggettivi.

Poiché tutti gli skeleton sono composti da `Computable` e poiché essi stessi sono dei `Computable`, è possibile annidare gli skeleton fra di loro grazie ai meccanismi di ereditarietà e polimorfismo messi a disposizione da C++.

3.3.2 Farm

Per creare una *farm* generica, è disponibile la funzione

```
template<typename T, typename V, V*(*fun)(T*)>
        Farm* createStandardFarm()
```

con `T` e `V` sottoclassi di `Task`. I worker della *farm* creata calcoleranno la funzione `fun` su ogni elemento ricevuto. Lo stream di ingresso dovrà produrre oggetti di tipo `T*` e lo stream di uscita dovrà gestire degli oggetti di tipo `V*`.

In alternativa è possibile costruire una farm utilizzando il costruttore `Farm (skel::Computable*)` e passando ad esso un'istanza del worker. In tal caso il comportamento dei worker va specificato all'interno del metodo `compute` della classe che definisce il worker.

Il motivo per cui non è necessario specificare emettitore e collettore è legato alla struttura dei grafi macro data flow. Ricordiamo che, per ogni task sullo stream di ingresso, viene creata una nuova istanza del grafo e quindi una nuova istanza del worker. Dato che il supporto per la gestione dei MDFg si occupa di suddividere fra i vari thread le MDFi da eseguire, questo è equivalente a suddividere i task fra i vari worker. Il supporto a tempo di esecuzione dei grafi MDFg si occupa, inoltre, di riordinare i task in modo corretto, facendo quindi le veci del collettore.

3.3.3 Pipeline

La libreria fornisce soltanto meccanismi per creare *pipeline* con due stadi. Per creare *pipeline* con un numero maggiore di stadi basta annidare più *pipeline* fra di loro. In alternativa, sarebbe stato possibile creare direttamente una *pipeline* con n stadi passando al costruttore un vettore di `computable`. Il motivo per cui è stata fatta questa scelta è quello di mantenere l'interfaccia quanto più simile possibile a quella di Muskel.

Per creare una *pipeline* generica è possibile utilizzare la funzione

```
template<typename T, typename U, typename V,
        U*(*stage1)(T*), V*(*stage2)(U*)>
        skel::Pipeline* createStandardPipeline()
```

con `T`, `U` e `V` sottoclassi di `Task`. La *pipeline* creata deve ricevere elementi di tipo `T*` e restituire elementi di tipo `V*`, con `U*` tipo dei valori intermedi.

In alternativa è possibile creare una *pipeline* definendo in modo esplicito i suoi stadi e passandoli al costruttore

```
Pipeline(skel::Computable *stage1, skel::Computable *stage2)
```

L'esempio 3.6 mostra la costruzione di una *pipeline* a tre stadi utilizzando un approccio misto. Questa *pipeline* calcola per ogni elemento dello stream la funzione $\sqrt{sin(cos(x))}$. Il primo e il secondo stadio vengono utilizzati come

stadi di una *pipeline* predefinita (linea 32), utilizzando le funzioni f e g (linee 4-9 e 11-16). Il terzo stadio (linee 18-28), invece, è creato in modo esplicito (linea 34) e in seguito annidato alla *pipeline* già creata (linea 36). I due approcci (utilizzo di una funzione o definizione di una classe) sono equivalenti dal punto di vista delle prestazioni. L'unica differenza è che mentre nel primo caso gli stadi della *pipeline* si limitano ad invocare la funzione sugli elementi ricevuti, nel secondo caso è possibile avere personalizzare il comportamento dello stadio (ad esempio l'oggetto può aggiornare il suo stato, ecc...).

Codice 3.6: Creazione di una pipeline.

```
1  #include <Skeleton/Computable.hpp>
2  #include <Skeleton/Pipeline.hpp>
3
4  DoubleTask* f(IntTask* iw){
5       DoubleTask* dw=new DoubleTask();
6       dw->set(std::cos(iw->get()));
7       delete iw;
8       return dw;
9  }
10
11 DoubleTask* g(DoubleTask* dw){
12      DoubleTask* toReturn=new DoubleTask();
13      toReturn->set(std::sin(dw->get()));
14      delete dw;
15      return toReturn;
16 }
17
18 class ThirdStage: public skel::Computable{
19 public:
20      Task** compute(Task** t){
21           DoubleTask* dw=(DoubleTask*) t;
22           Task** toReturn=new Task*[1];
23           toReturn[0]=new DoubleTask();
24           toReturn[0]=std::sqrt(dw->get());
25           delete dw;
26           return toReturn;
27      }
28 }
29
30 int main(){
31      //Creates a standard pipeline
32      Pipeline* p1=createStandardPipeline<IntTask,DoubleTask,
    DoubleTask,f,g>()
```

```
33      //Creates a custom stage
34      ThirdStage* x=new ThirdStage();
35      //Nests them together
36      Pipeline* p=new Pipeline(p1,x);
37      ...
38      ...
39  }
```

3.3.4 Map e reduce

Per quanto riguarda questi due skeleton, è stata definita un'unica classe che modelli il comportamento *emettitore-worker-wollettore* di uno skeleton data parallel; questa classe è `skel::EmitterWorkerCollector`. Il motivo per cui si è scelto di non trattare *map* e *reduce* in modo separato è dovuto al fatto che entrambi questi skeletons hanno la stessa struttura e quindi sono rappresentati dallo stesso grafo macro data flow. L'unica differenza nei due casi è relativa al comportamento degli elementi da cui sono composti.

La libreria mette a disposizione dei metodi per creare delle *map* e delle *reduce* che gestiscono degli `ArrayWrapper<T>`. `ArrayWrapper<T>` è una semplice classe generica definita in *Task.hpp*, introdotta per modellare task che rappresentano vettori con componenti di tipo `T`. Il comportamento di default degli emettitori è quello di creare dei sottovettori copiandovi dentro gli elementi del vettore ricevuto dall'input stream. In alternativa, è possibile specificare la macro `NOCOPY`, in modo tale che gli emettitori anziché creare dei nuovi sottovettori, inviino ad ogni worker una coppia `<Puntatore al vettore, Indici>`, dove gli indici servono ad indicare al worker i limiti del suo sottovettore.

Per creare una *map* con `nw` worker è possibile utilizzare la funzione

```
template<typename T, typename V, V*(*fun)(T*)>
        EmitterWorkerCollector* createStandardMap(int nw)
```

specificando come parametro template la funzione `fun` che verrà calcolata dai worker su tutte le componenti dei sottovettori ricevuti dall'emettitore. Lo stream di input deve produrre elementi di tipo `ArrayWrapper<T*>` e l'output stream deve trattare elementi di tipo `ArrayWrapper<V*>`, con `T` e `V` sottoclassi di `Task`.

Per creare una *reduce* generica è stata messa a disposizione la funzione

```
template<typename T, T*(*fun)(T*,T*)>
        EmitterWorkerCollector* createStandardReduce(int nw)
```

Questa funzione restituisce una *reduce* con `nw` worker, i quali calcoleranno la riduzione utilizzando la funzione binaria, commutativa ed associativa `fun`, passata come parametro del template. In questo caso lo stream di input deve generare elementi di tipo `ArrayWrapper<T*>` e l'output stream deve gestire elementi di tipo `T*`, con `T` sottoclasse di `Task`. In questo caso, i worker

Le funzioni appena introdotte, utilizzano emettitori, worker e collettori generici. Nel caso in cui si vogliano gestire collezioni diverse dai vettori non è possibile utilizzare i parametri generici e quindi bisogna definirli manualmente. Ad esempio, per risolvere il problema mostrato nell'equazione 1.1, è necessario creare una *map* e definire una classe che rappresenti le matrici. Per creare una *map* in modo esplicito bisogna costruire un emettitore che riceva sullo stream di ingresso le due matrici da moltiplicare, le suddivida in sottomatrici e le invii ai worker. In seguito va definito il worker, che deve moltiplicare le sottomatrici ricevute dall'emettitore ed inviarle al collettore. Infine si deve specificare il collettore, che deve ricomporre i risultati sommando le sottomatrici ricevute ed inviare sullo stream di uscita il risultato calcolato.

3.4 Compilatore

Il compilatore permette di trasformare uno skeleton in un MDFg equivalente, affinché esso sia eseguibile dall'interprete. La routine di compilazione è definita come una funzione ricorsiva invocabile tramite la funzione `Mdfg* compile(skel::Computable* c)`, dove `c` è lo skeleton da compilare. La funzione restituisce un puntatore ad un MDFg che rappresenta lo skeleton compilato.

Il passo base della funzione consiste nella compilazione dello skeleton sequenziale. Poiché non esiste esplicitamente il concetto di skeleton sequenziale, il compilatore considerà come tali tutti gli skeleton non conosciuti. In tal caso il grafo restituito è composto da un'unica MDFi che incapsula il calcolo sequenziale.

Come spiegato nella sezione 3.3.2, la farm non necessita di emettitore e collettore. Per questo motivo, la compilazione della farm consiste soltanto nella compilazione del suo worker.

Nel caso della pipeline, il compilatore innanzitutto compila i due stadi separatamente. Dopodiché collega l'ultima istruzione del primo stadio alla prima istruzione del secondo stadio e fa in modo che l'ultima istruzione del secondo stadio abbia come destinazione lo stream di output. In seguito inserisce nell'MDFg che rappresenta il primo stadio tutte le MDFi del secondo stadio, avendo cura di aggiornare in modo corretto gli identificatori. Infine

viene restituito un puntatore al grafo che rappresentava il primo stadio della pipeline.

Per quanto riguarda lo skeleton `EmitterWorkerCollector`, la compilazione avviene tramite i seguenti passi:

1. vengono create due MDFi corrispondenti ad emettitore e collettore e il worker viene compilato;
2. vengono create n copie delle MDFi del worker, con n numero di worker dello skeleton;
3. per ogni worker, l'emettitore viene collegato alla prima MDFi del worker e l'ultima MDFi del worker viene collegata al collettore;
4. il collettore viene modificato in modo che abbia come destinazione lo stream di output;
5. le MDFi dei worker e del collettore vengono aggiunte al MDFg dell'emettitore e gli identificatori vengono aggiornati in modo coerente;
6. viene restituito un puntatore al grafo che rappresentava l'emettitore.

3.5 Interprete

Lo scopo dell'interprete è quello di ricevere dal manager le MDFi da eseguire, di inviarle all'acceleratore di FastFlow e di ottenere i risultati.

Per creare un istanza dell'interprete bisogna invocare il costruttore `mdf::Interpreter(int parDegree)`, dove `parDegree` rappresenta il numero di worker da attivare nella farm. Per eseguire una MDFi il manager invocherà su un'istanza dell'interprete il metodo `void exec(mdf::Mdfi*)`. L'interprete a questo punto provvederà ad inviare la MDFi all'acceleratore che, utilizzando l'emettitore della farm, invierà l'istruzione ad un worker utilizzando una coda *SPSC lock-free*. A questo punto il worker invocherà il metodo `compute` della MDFi ed inserirà in una coda condivisa con l'interprete i risultati calcolati. Nel momento in cui il manager vorrà ottenere i risultati, dovrà invocare sull'istanza dell'interprete il metodo `wait`, che provvederà a svuotare le code condivise con i worker e a restituire i risultati al manager. Rispetto alle code trattate nella sezione 2.2, le code condivise fra interprete e worker sono implementate utilizzando delle liste piuttosto che dei buffer. L'interazione fra manager ed interprete è mostrata in figura 3.2,

Inizialmente l'interprete era stato realizzato utilizzando un approccio diverso. I worker, infatti, anziché condividere le code direttamente con l'interprete, inviavano i risultati calcolati al collettore. L'interprete, al momento

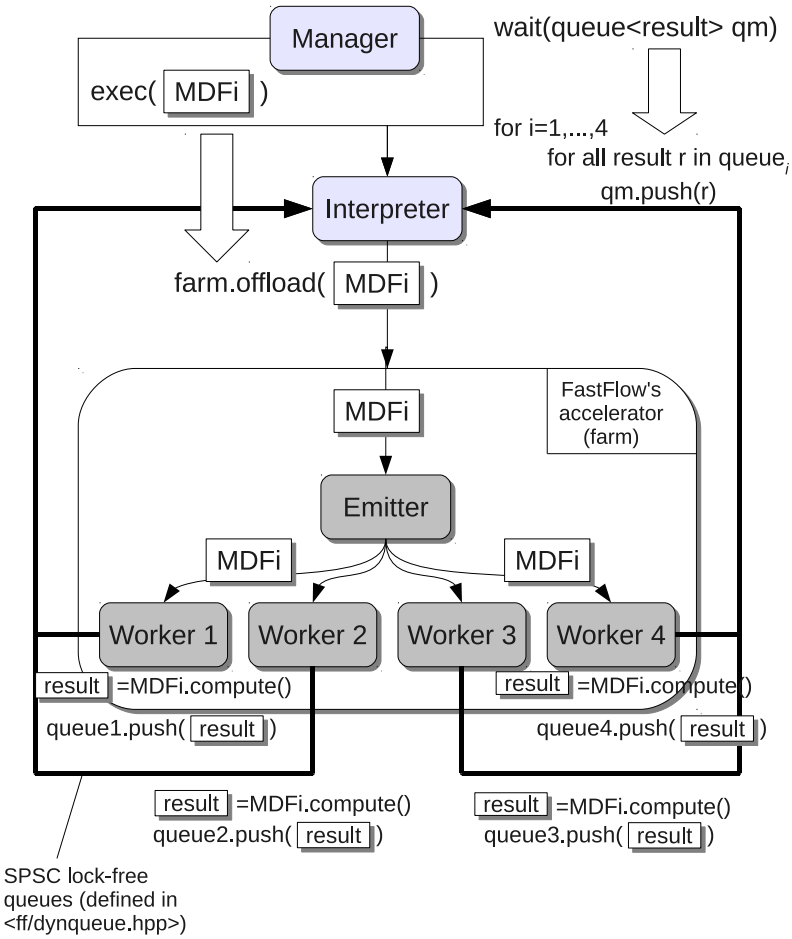

Figura 3.2: Interazione manager-interprete

dell'invocazione del metodo `wait`, provvedeva a riceve i risultati dal canale in uscita del collettore utilizzando il metodo non bloccante `load_result_nb`. Eliminando il collettore si è eliminato un thread, riuscendo quindi ad evitare molti cambi di contesto, con un conseguente aumento delle prestazioni nelle computazioni a grana fine.

3.6 Manager

Il manager si occupa della creazione delle istanze dei MDFg, dell'invio delle MDFi all'interprete, dell'aggiornamento dello stato dei MDFg, del riordinamento dei risultati e dell'invio di essi sullo stream di output.

Per creare un'istanza del manager bisogna invocare il costruttore

```
Manager(muskel::InputStream *i, muskel::OutputStream *o,
        int parDegree, int groupSize, skel::Computable *c)
```

con `i` e `o` che rappresentano rispettivamente lo stream di ingresso e quello di uscita e con `c` che rappresenta lo skeleton da eseguire. Il manager compila lo skeleton, crea un'istanza dell'interprete con grado di parallelismo pari a `parDegree`, ed invia ad esso al più `groupSize` MDFi per volta, prima di attendere i risultati dei calcoli. Esiste anche un altra versione del costruttore del manager, in cui è possibile specificare direttamente il MDFg anziché lo skeleton. Questa versione permette di eseguire MDFg non derivabili dalla compilazione di skeleton standard.

Per iniziare l'esecuzione basta invocare su un'istanza del manager il metodo `exec`. Il funzionamento del manager è illustrato in figura 3.3. Le seguenti istruzioni sono ripetute fino all'arrivo dell'*end-of-stream*:

1. se il numero di MDFi inviate all'interprete è minore di `groupSize` e ci sono MDFi *fireable* disponibili, allora queste vengono inviate all'interprete;

2. il manager legge un task dallo stream di input e crea una nuova istanza del MDFg da eseguire. Dopodiché estrae da questa istanza la prima MDFi e la invia all'interprete. Questo passo viene eseguito fin quando il numero di MDFi inviate all'interprete è minore di `groupSize` e finché ci sono altri task disponibili sullo stream;

3. ottiene il vettore dei risultati calcolati dall'interprete;

4. se il vettore è vuoto, l'esecuzione salta al punto 5. Altrimenti:

   4.1 estrae dal vettore un risultato e dal risultato estrae la destinazione del token;

   4.2 se la destinazione è lo stream di uscita, inserisce il risultato fra quelli da inviare sullo stream di uscita e ritorna al punto 4.1;

   4.3 altrimenti, estrae la MDFi destinataria del token e aggiorna il suo stato aggiungendo il nuovo token. Se l'inserimento del nuovo token ha reso *fireable* l'istruzione, essa viene aggiunta in fondo alla coda delle istruzioni *fireable*. In ogni caso l'esecuzione ritorna al punto 4.

5. invia in ordine sullo stream di uscita i risultati disponibili e ritorna al punto 1.

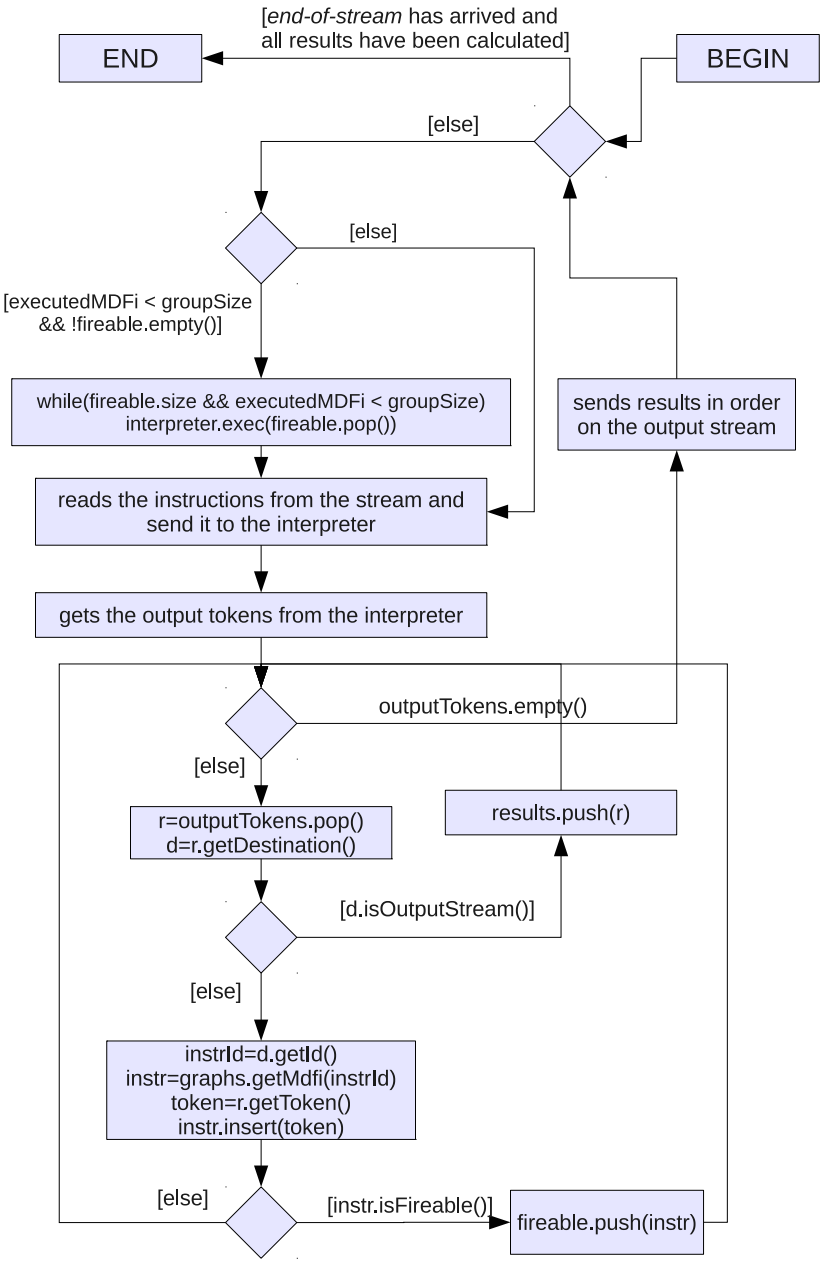

Figura 3.3: Funzionamento del manager

A questo punto è possibile introdurre brevemente le strutture dati utilizzate per la gestione dell'esecuzione e i motivi per cui sono state preferite rispetto ad altre.

Innanzitutto, analizziamo le operazioni da effettuare per la gestione dei MDFg. Per ogni task ricevuto dallo stream, bisogna creare una nuova istanza del grafo, quindi bisogna utilizzare una struttura dati che permetta di inserire elementi in modo efficente. Inoltre, poiché bisogna aggiornare spesso lo stato dei token all'interno delle istanze del grafo, questa struttura dati deve fornire buoni meccanismi di ricerca.

Tabella hash La struttura dati utilizzata per la gestione delle istanze dei MDFg è una tabella hash che utilizza liste di collisione [24], definita in *HashMap.hpp*. La tabella è composta da `HASHSIZE` liste di collisione, ognuna delle quali è un istanza delle liste standard del C++ (`std::list`). La funzione hash utilizzata è piuttosto semplice; per indicizzare il MDFg con identificatore i, infatti, bisogna accedere alla k-esima lista di collisione, con $k = i\%HASHSIZE$. In questa struttura dati, le operazioni di inserzione hanno sempre costo $O(1)$ poiché gli elementi vengono inseriti in fondo alla lista opportuna. Dato che i MDFg vengono generati utilizzando identificatori consecutivi, questo significa che le liste di collisione crescono di un elemento ogni `HASHSIZE` elementi ricevuti dallo stream di ingresso. Utilizzando un valore opportuno di `HASHSIZE`, quindi, è possibile effettuare le operazioni di indicizzazione con costo $O(1)$.

Pool Per ridurre l'impatto delle frequenti allocazioni/deallocazioni dinamiche dovute alla creazione e distruzione dei MDFg, è stata aggiunta la possibilità di utilizzare un pool definendo la macro `POOL`. Questa scelta nasce dalla considerazione del fatto che le istanze dei MDFg sono tutte uguali fra di loro, eccezion fatta per il loro identificatore e lo stato dei token. Anziché creare un nuovo grafo per ogni task ricevuto, è possibile riutilizzare le istanze dei MDFg create in precedenza e non più utilizzate, a condizione che l'identificatore venga aggiornato in modo corretto e che lo stato dei token venga riportato allo stato iniziale. Quindi, quando arriva un nuovo task sullo stream di input, è possibile utilizzare un MDFg estratto dal pool, piuttosto che creare una nuova istanza. Nello stesso tempo, anziché distruggere un grafo già utilizzato, lo si può inserire nel pool in modo che possa essere riutilizzato. Infine, è stata effettuata la scelta di limitare la dimensione del pool al valore della macro `MAXPOOLSIZE`, affinché esso non cresca a dismisura.

Istruzioni fireable Per quanto riguarda le istruzioni *fireable*, le uniche operazioni da effettuare sono inserzione e rimozione. Inoltre, non è necessario alcun tipo di ordinamento; per questi motivi si è scelto di utilizzare la `std::queue` messa a disposizione da C++.

Risultati calcolati Infine, bisogna inviare i risultati in modo ordinato sull'output stream. Poiché non è possibile prevedere l'ordine dei risultati calcolati, è stato necessario l'utilizzo di una struttura dati in cui poter inserire ed indicizzare elementi in modo molto efficente. Anche in questo caso la scelta è ricaduta sulla tabella hash definita in *HashMap.hpp*.

Interazione manager/interprete L'interazione fra manager ed interprete avviene in modo non bloccante. In questo modo, nel caso in cui non ci siano risultati disponibili, il manager può continuare a svolgere il suo compito continuando a leggere task dallo stream di input e ad inviare sullo stream di uscita i risultati già calcolati. Utilizzando questa soluzione, il manager resta attivo per tutto il tempo dell'esecuzione e quindi occupa uno dei core del sistema. La conseguenza di questa scelta è che la farm di FastFlow non può effettivamente sfruttare tutti i core messi a disposizione, poiché altrimenti si verificherebbe un degrado delle prestazioni dovuto ai numerosi cambi di contesto fra il thread che rappresenta il manager e uno dei thread della farm.

L'alternativa, adottata inizalmente, consisteva nel bloccarsi nell'attesa di tutti i risultati inviati (cioè al più `groupSize`). Se da un lato questo permetteva alla farm di FastFlow di sfruttare tutti i core, dall'altro portava molti svantaggi. Per indicare alla farm che si era intenzionati ad attendere i risultati, bisognava inviare un *end-of-stream* in modo da bloccarla per poi riavviarla in seguito, con un conseguente aumento dei tempi di calcolo dovuto alla distruzione e alla successiva creazione dei thread della farm. Se la dimensione dello stream è pari ad n elementi, questo portava a riavviare la farm $\frac{n}{groupSize}$ volte. Per ridurre l'impatto quindi ci sarebbe stato bisogno di scegliere un valore molto alto per `groupSize`, in modo da attendere un numero maggiore di risultati e diminuire il numero di ''riavvii'' della farm. Questa scelta però è svantaggiosa poiché non permette di inviare un po' per volta i risultati calcolati sullo stream di uscita.

Per questi motivi, si è preferito ''rinunciare'' ad un core ed avere un'interazione asincrona fra manager ed interprete.

Capitolo 4

Risultati sperimentali

In questo capitolo verranno analizzate le prestazioni di Faskel utilizzando alcune applicazioni appositamente sviluppate. Gli esperimenti riportati sono stati eseguiti su due piattaforme diverse, presenti nel laboratorio di *architetture parallele* del nostro dipartimento.

Andromeda macchina Intel con 2 quad-core Xeon E5520 Nehalem (16 HyperThreads) @2.26GHz. Questa macchina ha una cache L3 da 8MB e 24 GBytes di memoria;

Ottavinareale macchina Intel con 2 quad-core Xeon E5420 Harpertown @2.5GHz con una cache L2 da 6MB e 8 GBytes di memoria.

4.1 Test funzionalità

Per essere sicuri che la libreria funzioni correttamente, è stata sviluppata una piccola classe di test che verifichi la funzionalità delle caratteristiche della libreria. Questa classe è definita nel file */examples/skeletonTest.hpp* e permette di verificare la correttezza di:

- compilazione degli skeleton forniti;
- composizione degli skeleton;
- gestione dell'esecuzione da parte di manager ed interprete;
- gestione degli stream (sia sincroni che asincroni);

Inoltre, per verificare l'assenza di *leak* di memoria[1] è stato utilizzato il tool *Valgrind* [25].

4.2 Test scalabilità

Per effettuare i test di scalabilità è stato utilizzato un programma, definito in *examples/dummy.cpp*, che utilizza una *farm*. I worker di questa *farm*, una volta ricevuto un valore di tipo `double`, eseguono su di esso n iterazioni della funzione `std::sin` di C++. Variando il numero di iterazioni è possibile variare il tempo speso all'interno dei worker e di conseguenza analizzare la scalabilità della libreria in funzione della grana computazionale.

Ricordiamo che la grana computazionale è una quantità definita come

$$Grain = \frac{T_c}{T_{com}}$$

dove T_{com} è il tempo impiegato per trasmettere un task al worker e per riceverne il risultato, e T_c è il tempo impiegato dal worker per eseguire un task.

Per calcolare T_{com} è possibile definire la macro `COMPUTE_COM_TIME`. Utilizzando questa macro, è possibile utilizzare *dummy.cpp* per effettuare il calcolo del tempo di comunicazione. Nel nostro caso, eseguendo il calcolo su `andromeda`, abbiamo ottenuto un valore di $T_{com} \simeq 2\mu s$.

A questo punto il calcolo di T_c è piuttosto semplice poiché basta calcolare la differenza fra il momento in cui si entra nel worker e quello in cui viene restituito il risultato.

I primi test effettuati hanno avuto come obiettivo l'analisi dello speedup di Faskel al variare della grana computazionale. Lo speedup è una quantità definita come

$$S_p = \frac{T_1}{T_p}$$

dove p è il numero di processori, T_1 è il tempo di completamento della versione sequenziale dell'algoritmo e T_p è il tempo di completamento della versione parallela con p processori. Per calcolare il tempo di completamento della versione sequenziale bisogna invocare il programma `dummy` con 0 worker.

In figura 4.1 possiamo vedere come lo speedup aumenti all'aumentare della grana computazionale.

[1] Per *leak* si intende la mancata deallocazione di memoria allocata dinamicamente. Questo problema può portare ad una crescita smisurata della memoria occupata e di conseguenza ad un probabile blocco del sistema.

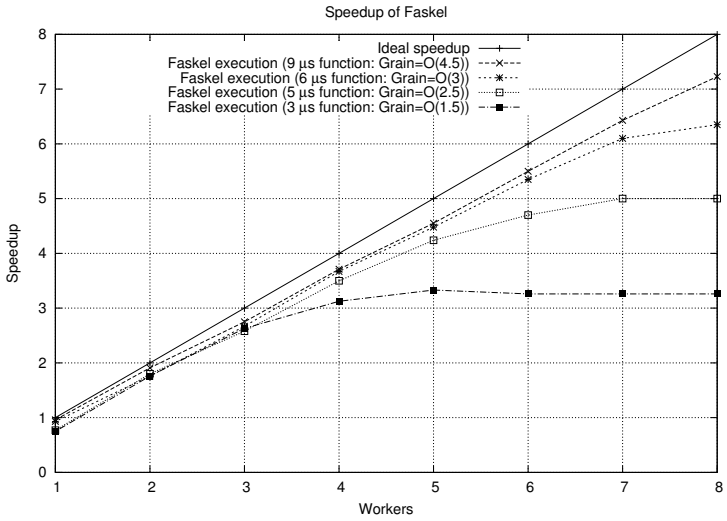

Figura 4.1: Speedup di Faskel

In figura 4.2 sono presentati gli stessi risultati ma relativi all'efficenza al variare della grana e del numero di worker utilizzati. L'efficenza è definita come

$$E_p = \frac{S_p}{p}$$

con p numero di processori utilizzati. L'efficenza è una quantita compresa tra 0 e 1, che permette di valutare quanto tempo viene speso in comunicazioni e sincronizzazioni. Quindi, se il valore di questo parametro si avvicina a 0, significa che il tempo speso nelle comunicazioni ha un forte impatto.

Da questi risultati possiamo vedere come l'efficienza aumenta all'aumentare della grana e che nel caso in cui vengono utilizzati pochi worker l'efficienza riesce comunque ad avere valori elevati anche in presenza di grana fine.

4.3 Accorgimenti adottati al fine di aumentare le prestazioni

In questa sezione metteremo in evidenza gli accorgimenti adottati al fine di aumentare le prestazioni. Per effettuare questi test è stato utilizzato il programma `dummy`, definito in */examples/dummy.cpp*.

Funzioni inline L'utilizzo delle funzioni `inline` ha permesso di evitare il costo dell'allocazione dei parametri sullo stack per molte delle funzioni

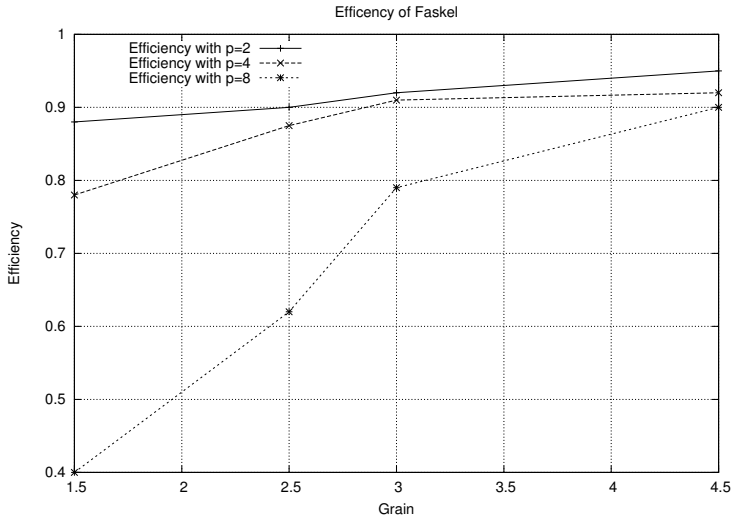

Figura 4.2: Efficienza di Faskel al variare della grana e del numero di worker

utilizzate. In figura 4.3 possiamo vedere come questo piccolo accorgimento contribuisce all'aumento delle prestazioni, soprattutto all'aumentare del numero di *worker* utilizzati. Il motivo per cui il grafico ha un comportamento ''a forbice'' è dovuto al fatto che all'aumentare dei *worker*, aumenta il numero di risultati calcolati per intervallo di tempo. In questo caso un piccolo ritardo da parte del manager può portare un notevole aumento nelle dimensioni del canale di uscita dell'acceleratore e di conseguenza ad un degrado delle prestazioni.

Pool di grafi Come descritto nella sezione 3.6, è possibile utilizzare un pool per la gestione delle istanze dei MDFg. I risultati ottenuti sono mostrati in figura 4.4 e anche in questo caso il vantaggio è evidente soprattutto all'aumentare del numero di worker utilizzati. I motivi per cui questo accade sono gli simili a quelli visti in precedenza. Infatti, se i risultati arrivano ad intervalli più ravvicinati, i costi delle allocazioni e deallocazioni non vengono più ''ammortizzate'' dalle altre operazioni. Inoltre possiamo vedere come all'aumentare dei worker utilizzati, la soluzione che non utilizza il pool non riesce più a scalare.

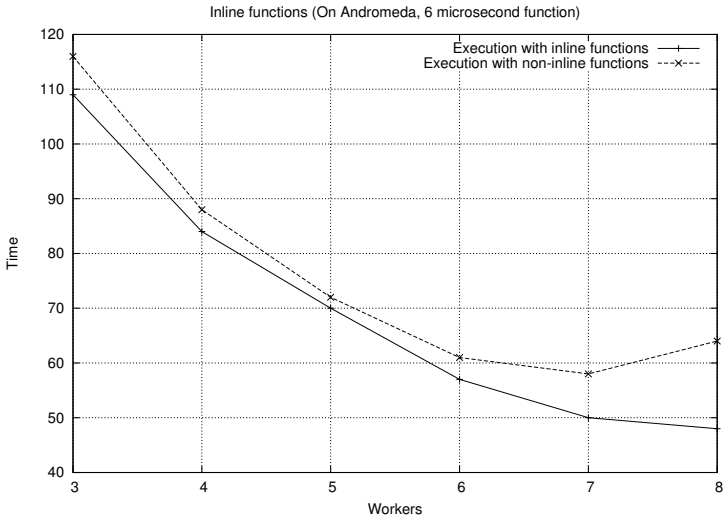

Figura 4.3: Aumento di prestazioni dovuto all'utilizzo di funzioni inline

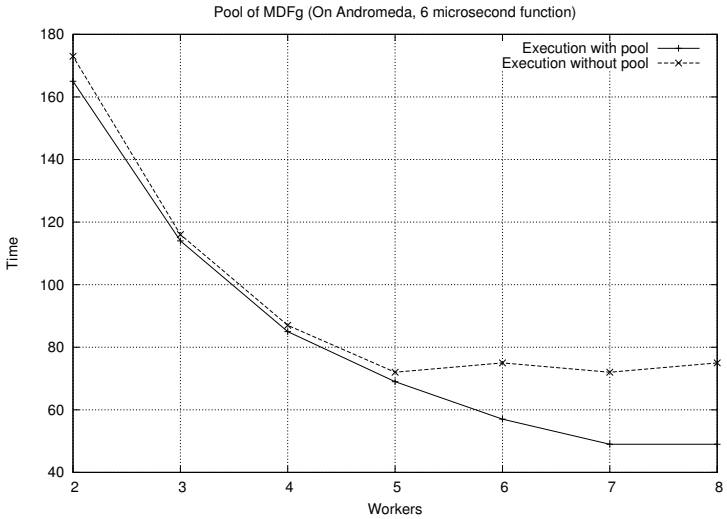

Figura 4.4: Aumento di prestazioni dovuto all'utilizzo del pool di MDFg

4.4 ffProbe: realizzazione di una cache NetFlow

Oltre alle classi di test appena viste, è stata realizzata una cache NetFlow [6]. Originariamente questa cache era stata realizzata come progetto finale dell'esame di *Gestione di reti* utilizzando FastFlow ed in seguito è stata effettuata una conversione in modo da poter utilizzare Faskel come supporto dell'esecuzione.

Flussi Una cache NetFlow è un analizzatore di traffico di rete che aggrega i pacchetti in flussi. Un flusso è un insieme di pacchetti con lo stesso valore per i campi:

- indirizzo IP sorgente;
- indirizzo IP destinazione;
- protocollo a livello 4;
- porta sorgente del protocollo a livello 4;
- porta destinazione del protocollo a livello 4;
- IP TOS.

Per ogni flusso, utilizzando come chiave i 6 valori appena introdotti, è possibile accedere alla cache ed ottenere informazioni su di esso. Queste informazioni consistono in:

- eventuali flag TCP ricevuti;
- numero di pacchetti appartenenti al flusso;
- numero di byte appartenenti al flusso;
- istante in cui è stato ricevuto il primo pacchetto;
- istante in cui è stato ricevuto l'ultimo pacchetto;
- informazioni di routing (non presenti in ffProbe):
  - sistema autonomo di origine e di destinazione;
  - interfaccia su cui si è ricevuto il pacchetto;
  - maschera della sottorete di origine e di quella di destinazione;

  - indirizzo IP del prossimo *hop*.

I flussi vengono mantenuti nella cache fino al momento in cui non diventano scaduti. Si dice che un flusso è scaduto se:

- arriva il flag `FIN` o `RST` (è possibile solo se si tratta di un flusso TCP);
- il flusso è inattivo per un certo intervallo di tempo (tipicamente 15 secondi);
- il flusso è rimasto attivo per troppo tempo (tipicamente 30 secondi).

Ogni volta che un flusso scade, esso viene inserito in un coda e successivamente inviato ad un *collettore* remoto. Lo scopo del collettore è quello di gestire le statistiche sui flussi e dedurre alcune da esse se ci sono attacchi in corso o se alcuni utenti hanno in esecuzione dei programmi per il *peer-to-peer*.

Utilizzo di ffProbe Il programma per essere compilato necessita della libreria *libpcap* [26] installata sul sistema. I comandi di base per utilizzare ffProbe sono:

- *make install* installa ffProbe sul sistema;
- *make uninstall* rimuove ffProbe dal sistema;
- *./ffProbe -h* stampa informazioni sui parametri che è possibile specificare.

I parametri utilizzabili non sono stati riportati per non appesantire troppo l'esposizione.

4.4.1 Struttura con FastFlow

Per quanto riguarda la struttura, lo skeleton utilizzato è una *pipeline* (la cui struttura è rappresentata in figura 4.5) composta da tre tipi diversi di stadi. Il primo stadio (*Reader*) si occupa di leggere i pacchetti dal socket pcap e di estrarre da essi le informazioni utili alla generazione dei flussi. Per ogni lettura dal socket questo stadio genera un task che invierà al secondo stadio (*Worker*). In questo caso, un task contiene una lista di flussi scaduti e n liste di pacchetti, dove n è il numero dei worker e dove la lista i-esima contiene i pacchetti destinati al worker i-esimo. Il reader decide in quale lista inserire i pacchetti catturati a seconda del valore della funzione hash calcolata su quest'ultimi.

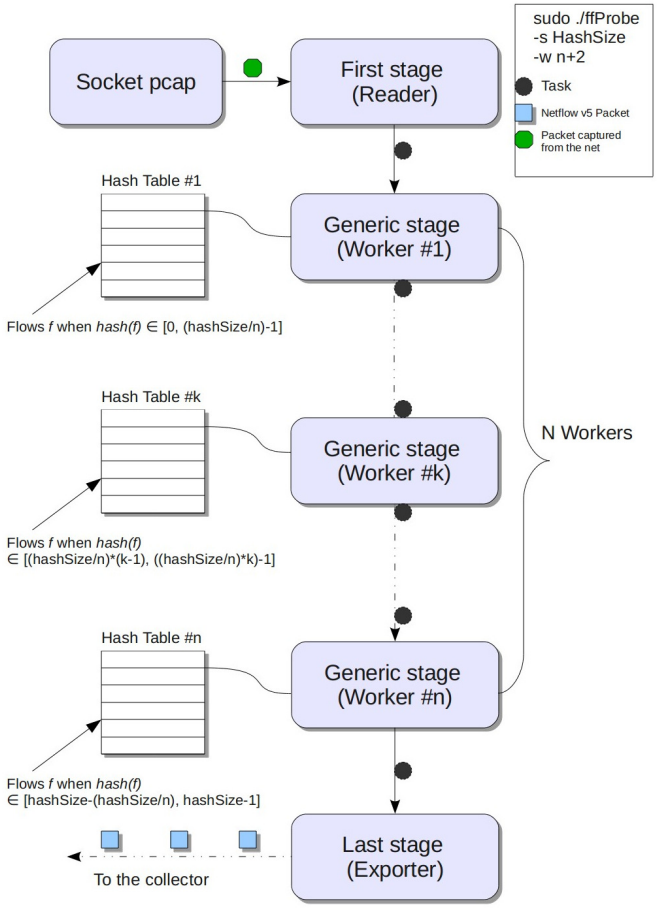

Figura 4.5: Struttura di ffProbe

La cache è implementata come una tabella hash e ciascun worker si occupa di gestire una porzione della tabella totale. In realtà ogni worker ha una propria tabella hash in modo tale che i thread non condividano alcun dato fra di loro (tranne ovviamente le code lock-free per la comunicazione fra i vari stadi della pipeline e l'allocatore di fastflow). Il worker quindi una volta ricevuto il task, controlla il numero k di flussi destinati alla sua tabella hash; se k è uguale a 0 allora controllerà la scadenza di alcuni flussi utilizzando una strategia *round-robin*[2]. Se k è diverso da 0 il worker aggiornerà i flussi della tabella hash e controllerà anche in questo caso la scadenza di alcuni flussi. In entrambi i casi i flussi scaduti vengono aggiunti alla lista nel task. È possibile specificare il numero di flussi di cui controllare la scadenza utilizzando alcuni parametri di configurazione.

Infine il task viene inviato sul canale di comunicazione verso lo stadio successivo (che può essere un altro worker oppure l'ultimo stadio della pipeline). Una volta che il task arriva all'ultimo stadio (*Exporter*), i flussi scaduti vengono aggiunti ad una coda privata. Quando la coda contiene 30 flussi o quando scade il timer specificato nei parametri di configurazione, i flussi nella coda vengono inseriti in un record *NetFlow v5* e spediti al collector remoto (eventualmente vengono anche salvati su un file in formato testuale).

I task e le liste di pacchetti da aggiungere alle tabelle hash vengono allocati con l'allocatore di FastFlow (nella versione che permette un solo allocatore) poiché una volta create vengono modificate dagli altri thread soltanto per rimuovere elementi.

La lista di flussi scaduti viene invece allocata con l'allocatore standard poiché ogni *worker* può aggiungere a questa lista nuovi elementi. Oltre all'utilizzo dell'allocatore di FastFlow sono stati adottati altri accorgimenti per migliorare le prestazioni complessive:

- alle funzioni i parametri sono stati passati per copia di puntatori anziché per copia dei valori puntati. Questo permette di ridurre il costo dell'inserimento dei parametri sullo stack;

- è stato fatto un massiccio uso di funzioni `inline` in modo da evitare l'inserimento di parametri sullo stack durante l'invocazione di alcune funzioni;

- la conversione dei valori da network byte order ad host byte order è effettuata soltanto nel momento in cui i flussi vengono (eventualmente) salvati su file;

[2] La strategia round-robin consiste nel controllare i flussi in modo consecutivo e circolare.

- come accennato in precedenza non c'è nessun dato condiviso fra i thread e non c'è nessun meccanismo di lock (tranne quello previsto dall'allocatore standard di g++).

4.4.2 Struttura con Faskel

La versione scritta utilizzando Faskel è molto simile alla precedente, con l'unica differenza che il thread *reader* e il thread *exporter* sono stati rimpiazzati rispettivamente con l'input stream e con l'output stream. In questo caso quindi le funzioni calcolate da questi due thread vengono ora calcolate dal manager di Faskel.

4.4.3 Test

Per verificare la funzionalità del programma sono stati eseguiti diversi test su alcuni file *.pcap* e sono stati confrontati i risultati con quelli ottenuti facendo processare gli stessi file dalla cache commercialie nProbe [27]. I risultati ottenuti nei due casi coincidono perfettamente. Il programma inoltre è stato controllato con il tool Valgrind [25] per verificare l'assenza di *leak* di memoria. Infine è stato utilizzato Wireshark [28] per controllare la correttezza del formato dei pacchetti esportati verso il collector.

Purtroppo, per ragioni di tempo e di hardware a disposizione, non è stato possibile effettuare dei test prestazionali.

Capitolo 5

Conclusioni e sviluppi futuri

Lo scopo di questo tirocinio era la reimplementazione di Muskel, mantenendone invariata la semantica ma fornendo la possibilità di eseguire i grafi macro data flow su macchine multicore a memoria condivisa. Per rendere possibile ciò, l'interprete di Muskel è stato realizzato utilizzando FastFlow, una libreria scritta in C++ che fornisce meccanismi efficenti per la comunicazione fra thread in modo non bloccante.

Poiché gli strumenti da utilizzare erano una parte fondamentale dei requisiti del tirocinio, la prima parte di esso è stata investita nello studio e nell'utilizzo di tali strumenti.

L'obiettivo iniziale del tirocinio era di creare una libreria con un interfaccia equivalente a quella di Muskel, che permettesse l'utilizzo degli skeleton *farm* e *pipeline* e delle composizioni arbitrarie fra di essi. La libreria doveva inoltre fornire la possibilità di definire dei grafi macro data flow e di interpretarli su macchine multicore. Tutti i requisiti iniziali sono stati rispettati e sono stati aggiunti in seguito gli skeleton *map* e *reduce*. Inoltre, al fine di rendere la libreria utilizzabile anche per computazioni a grana fine, le strutture dati utilizzate sono state studiate ed implementate in modo da garantire una buona efficenza. Infine, oltre ad alcune piccole demo, è stata sviluppata un applicazione per l'analisi del traffico di rete, il cui funzionamento è mostrato nella sezione 4.4.

I risultati ottenuti hanno confermato le ipotesi iniziali, cioè che è possibile utilizzare FastFlow anche per computazioni a grana molto fine. Inoltre, come supposto inizialmente e come mostrato nella sezione 4.2, le prestazioni peggiorano al diminuire della grana computazionale.

Infine sono stati implementati alcuni meccanismi, mostrati nella sezione 4.3, che hanno permesso di riuscire ad avere una buona scalabilità anche dove ciò inizialmente non era possibile.

Sviluppi futuri Durante lo sviluppo della libreria sono state analizzate alcune caratteristiche che possono essere aggiunte alla libreria. Fra queste possiamo citare:

- gestione dello stream di ingresso in modo bloccante. Una possibile soluzione a questo problema potrebbe essere quella di utilizzare un thread addizionale che si occupi di gestire la lettura dei dati dallo stream. In questo modo il thread manager è indipendente e può continuare ad elaborare le MDFi anche in assenza di task sullo stream di ingresso;

- ottimizzazioni nella compilazione degli skeleton. Quando gli skeleton vengono composti fra di loro è possibile effettuare delle ottimizzazioni in modo da eliminare alcuni stadi o aggregarne altri fra di loro. Questo permette di ridurre le dimensioni dei MDFg ottenuti dal processo di compilazione e di conseguenza di aumentare le prestazioni;

- ottimizzazioni nella gestione delle strutture dati. La scelta delle strutture dati da utilizzare è stata molto mirata e questo ha permesso di aumentare notevolmente le prestazioni, soprattutto in computazioni a grana fine. Nonostante tutto è possibile ottimizzare ulteriormente queste strutture dati; ad esempio, nella gestione delle liste di collisione delle tabelle hash, è possibile utilizzare un *pool* di nodi, in modo da evitare molte allocazioni e deallocazioni dinamiche;

- analizzare le prestazioni di ffProbe. Questo permetterebbe di analizzare il funzionamento di Faskel in una situazione reale di una certa complessità. Inoltre paragonando le prestazioni della versione che utilizza FastFlow con la versione che utilizza Faskel è possibile capire qual'è l'overhead introdotto dalla gestione dei grafi macro data flow.

Bibliografia

[1] Wikipedia. Algorithmic skeleton --- Wikipedia, the free encyclopedia, 2010. [Online; accessed 20-August-2010].

[2] Marco Aldinucci, Marco Danelutto, and Patrizio Dazzi. Muskel: an expandable skeleton environment. *Scalable Computing: Practice and Experience*, 8(4):325--341, December 2007.

[3] Marco Danelutto. Muskel website, 2010. [Online; accessed 20-August-2010].

[4] Marco Aldinucci, Massimo Torquati, and Massimiliano Meneghin. FastFlow: Efficient parallel streaming applications on multi-core. Technical Report TR-09-12, Università di Pisa, Dipartimento di Informatica, Italy, September 2009.

[5] Università di Pisa e Torino. Fastflow: programming multicores, 2010. [Online; accessed 20-August-2010].

[6] Cisco. Netflow services solutions guide, 2010. [Online; accessed 20-August-2010].

[7] Murray Cole. *Algorithmic skeletons: structured management of parallel computation*. MIT Press, Cambridge, MA, USA, 1991.

[8] Rita Loogen, Yolanda Ortega-mallén, and Ricardo Pe na-marí. Parallel functional programming in eden. *J. Funct. Program.*, 15(3):431--475, 2005.

[9] Marco Vanneschi. The programming model of assist, an environment for parallel and distributed portable applications. *Parallel Comput.*, 28(12):1709--1732, 2002.

[10] Marco Aldinucci, Massimo Coppola, Marco Danelutto, Nicola Tonellotto, Marco Vanneschi, and Corrado Zoccolo. High level grid programming with ASSIST. *Computational Methods in Science and Technology*, 12(1):21--32, 2006.

[11] Bruno Bacci, Marco Danelutto, Salvatore Orlando, Susanna Pelagatti, and Marco Vanneschi. P3l: A structured high level programming language and its structured support. *Concurrency Practice and Experience*, 7(3):225--255, May 1995.

[12] Bruno Bacci, Marco Danelutto, Susanna Pelagatti, and Marco Vanneschi. Skie: A heterogeneous environment for hpc applications. *Parallel Computing*, 25(13-14):1827--1852, 1999.

[13] Murray Cole. Bringing skeletons out of the closet: a pragmatic manifesto for skeletal parallel programming. *Parallel Comput.*, 30(3):389--406, 2004.

[14] Anne Benoit Murray, Murray Cole, Stephen Gilmore, and Jane Hillston. Flexible skeletal programming with eskel. In *In: 11th Intl Euro-Par: Parallel and Distributed Computing, vol. 3648 of LNCS, 761-770, Lisbona*, pages 761--770. Springer-Verlag, 2005.

[15] Kiminori Matsuzaki, Hideya Iwasaki, Kento Emoto, and Zhenjiang Hu. A library of constructive skeletons for sequential style of parallel programming. In *InfoScale '06: Proceedings of the 1st international conference on Scalable information systems*, page 13, New York, NY, USA, 2006. ACM.

[16] H. Kuchen and J. Striegnitz. Features from functional programming for a c++ skeleton library: Research articles. *Concurr. Comput. : Pract. Exper.*, 17(7-8):739--756, 2005.

[17] Marco Aldinucci and Marco Danelutto. Skeleton based parallel programming: functional and parallel semantic in a single shot. *Computer Languages, Systems and Structures*, 33(3-4):179--192, October 2007.

[18] Bjarne Stroustrup. *Linguaggio, libreria standard, principi di programmazione*. Pearson Education Italia, 2000.

[19] B. W. Kernighan and D. M. Ritchie. *The C Programming Language*. Prentice Hall, 1978.

[20] Leslie Lamport. Specifying concurrent program modules. *ACM Trans. Program. Lang. Syst.*, 5(2):190--222, 1983.

[21] John Giacomoni, Tipp Moseley, and Manish Vachharajani. Fastforward for efficient pipeline parallelism: a cache-optimized concurrent lock-free queue. In *PPoPP '08: Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming*, pages 43--52, New York, NY, USA, 2008. ACM.

[22] Marco Aldinucci, Marco Danelutto, Massimiliano Meneghin, Peter Kilpatrick, and Massimo Torquati. Efficient streaming applications on multi-core with FastFlow: the biosequence alignment test-bed. In Barbara Chapman, Frédéric Desprez, Gerhard R. Joubert, Alain Lichnewsky, and Frans Peters anf Thierry Priol, editors, *Parallel Computing: From Multicores and GPU's to Petascale (Proc. of PARCO 2009, Lyon, France)*, volume 19 of *Advances in Parallel Computing*, pages 273--280, Lyon, France, September 2009. IOS press.

[23] Marco Aldinucci, Marco Danelutto, Peter Kilpatrick, Massimiliano Meneghin, and Massimo Torquati. Accelerating sequential programs using FastFlow and self-offloading. Technical Report TR-10-03, Università di Pisa, Dipartimento di Informatica, Italy, February 2010.

[24] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 2001.

[25] Valgrind home page, 2010. [Online; accessed 20-August-2010].

[26] Tcpdump/libpcap home page, 2010. [Online; accessed 20-August-2010].

[27] Luca Deri. nprobe: an open source netflow probe for gigabit networks. In *In Proc. of Terena TNC 2003*, 2003.

[28] Wireshark home page, 2010. [Online; accessed 20-August-2010].